

CYBERSECURITY ENHANCEMENT THROUGH MALICIOUS URL DETECTION WITH ML

A Major Project report was submitted
in partial fulfillment of the requirement for the award of the degree

BACHELOR OF TECHNOLOGY

in

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE

By

2103A51067

2103A51550

2103A51553

2103A51218

Rontala Pooja Reddy

Bathini Shiva Sathvika

Errabelly Shruthi

Challa Akshay

Under the guidance of

Mr. Ch. Sandeep

Associate Professor at the School of CS&AI.



SR UNIVERSITY

SR University, Ananthasagar, Warangal, Telangana-506371

SR University

Ananthasagar, Warangal.



CERTIFICATE

This is to certify that this project entitled “**Cybersecurity Enhancement Through Malicious URL Detection With ML**” is the bonafide work carried out by, **Rontala Pooja Reddy, Bathini Shiva Sathvika, Errabelly Shruthi, Challa Akshaya** as a Major Project for the partial fulfillment to award the degree **BACHELOR OF TECHNOLOGY** in **School of Computer Science and Artificial Intelligence** during the academic year 2024-2025 under our guidance and Supervision.

Mr.Ch. Sandeep

Associate Professor

SR University

Anathasagar, Warangal

Dr. M. Sheshikala

Professor & Head,

School of CS&AI,

SR University

Ananthasagar, Warangal.

Reviewer-1

Name:

Designation:

Signature:

Reviewer-2

Name:

Designation:

Signature:

ACKNOWLEDGEMENT

We owe an enormous debt of gratitude to our Major project guide Mr.**Ch.Sandeep, Assoc Prof** as well as Head of the Department of CS&AI, **Dr. M.Sheshikala, Professor** and Dean of the School of CS&AI, **Dr.Indrajeet Gupta Professor** for guiding us from the beginning through the end of the Major Project with their intellectual advice and insightful suggestions. We truly value their consistent feedback on our progress, which was always constructive and encouraging and ultimately drove us in the right direction.

We express our thanks to project coordinators **Mr. Sallauddin Md, Asst. Prof., and R. Ashok Asst. Prof.** for their encouragement and support.

Finally, we express our thanks to all the teaching and non-teaching staff of the department for their suggestions and timely support.

ABOUT THE ORGANIZATION

SR University (SRU), an autonomous private institution located in Warangal, Telangana, India, offers a variety of academic programs with an emphasis on interdisciplinary learning and practical, real-world application. The university provides both undergraduate and postgraduate courses, including B.E/B.Tech, MBA/PGDM, M.E/M.Tech, BBA, B.Sc, and M.Sc programs. SRU's curriculum encourages innovative thinking and the integration of theoretical concepts into practical solutions.

The university boasts excellent infrastructure, featuring a large library and a digital library, a sports complex, and modern hostels for both male and female students. SRU also offers flexible academic options such as a flexible credit system, opportunities for minor degrees and specializations, branch changes, integrated programs, and semester abroad plans. With a strong focus on research, SRU has over 50 sponsored projects and more than 190 patents to its name. The university's placement cell provides robust support for job placements and internship opportunities.

SRU also fosters extra-curricular engagement through clubs like dance and music clubs, which enable students to participate in various intra and inter-college competitions. Additionally, SRU's Ph.D. program is designed to provide comprehensive training in research methodology, technical communication, and literature review.

Abstract

Cybersecurity threats, such as phishing, malware, and other cyberattacks, are increasing as more people use the internet for various activities like shopping, banking, and communication. Many harmful websites try to steal personal information, install viruses, or trick users into providing sensitive data. To protect users from these dangers. This project Cybersecurity Enhancement through Malicious URL (Uniform Resource Locator) Detection with ML (Machine Learning) uses machine learning to identify and classify dangerous URLs effectively.

The system examines different features of a website's address URL to determine if it is safe, malicious, or suspicious. It checks factors like URL length, the use of special characters, domain age, the presence of numeric values, and popularity ranking. It analyzes the website structure, hosting details, SSL (Secure Sockets Layer) certificate validity, and redirection patterns to detect potentially harmful sites. Machine learning models such as Random Forest and Support Vector Machines (SVM) help make accurate predictions based on past data and continuously improve as they learn from new threats.

A real-time tool is included in this project allowing users to enter a website link and get immediate safety classifications along with confidence scores. The system provides insights into why a URL is flagged as suspicious helping users make informed decisions. To improve accuracy the system connects with external sources like Google Safe Browsing and WHOIS (A network protocol and database that provides information about domain name and IP address), which provide additional information about the website's reputation, domain registration details, and historical records.

By combining machine learning, smart feature analysis, and external data, this project creates a highly reliable way to detect harmful links. The goal is to improve cybersecurity for individuals and organizations, making the Internet a safer place for everyone. Future improvements may include integrating browser extensions, mobile applications, and AI-powered threat intelligence for real-time protection while browsing. The project aims to stay updated with the latest cyber threats to ensure continuous security enhancement.

TABLE OF CONTENTS

<u>Chapter No.</u>	<u>Page No.</u>
1. Introduction	1
2. Related Work	2
3. Problem Statement	4
4. Requirement Analysis, Risk Analysis, Feasibility Analysis	5
5. Proposed Plan	7
6. Advantages	8
7. Literature Survey	9
8. Architecture Design	24
8.1 Class Diagram of URL Guardian	24
8.2 Sequence Diagram of URL Guardian	24
8.3 Use-Case Diagram of URL Guardian	25
8.4 Activity Diagram of URL Guardian	26
8.5 Data Flow Diagram Level-0	27
8.6 Data Flow Diagram Level-1	27
8.7 Architecture Diagram of URL Guardian	28
9. Implementation	29
9.1 Malicious URL Detection Process	30
9.2 Malicious URL Detection System Architecture	32
9.3 Flow Diagram	33

10. Result	83
10.1 Home page of URL Guardian	83
10.2 Detect Page of URL Guardian	83
10.3 About Us Page of URL Guardian	84
10.4 Sign-In Page of URL Guardian	84
10.5 Register Page of URL Guardian	85
10.6 Test Case-1	85
10.7 Test Case-2	86
10.8 Test Case-3	86
10.9 Test Case-4	87
10.10 Test Case-5	87
10.11 Donut Graph of different URLs	88
10.12 Heat Map of Features Correlations	88
10.13 Confusion Matrix	89
11. Learning Outcomes	90
12. Conclusion with Challenges	91
13. References / Bibliography	92
14. Links / Web References	94

LIST OF FIGURES

<u>Figure No.</u>	<u>Page No.</u>
8.1 Class Diagram of URL Guardian	24
8.2 Sequence Diagram of URL Guardian	24
8.3 Use-Case Diagram of URL Guardian	25
8.4 Activity Diagram of URL Guardian	26
8.5 Data Flow Diagram Level-0	27
8.6 Data Flow Diagram Level-1	27
8.7 Architecture Diagram of URL Guardian	28
9.1 Malicious URL Detection Process	30
9.2 Malicious URL Detection System Architecture	32
9.3 Flow Diagram	33
10.1 Home page of URL Guardian	83
10.2 Detect Page of URL Guardian	83
10.3 About Us Page of URL Guardian	84
10.4 Sign-In Page of URL Guardian	84
10.5 Register Page of URL Guardian	85
10.6 Test Case-1	85
10.7 Test Case-2	86
10.8 Test Case-3	86
10.9 Test Case-4	87
10.10 Test Case-5	87
10.11 Donut Graph of different URLs	88
10.12 Heat Map of Features Correlations	88
10.13 Confusion Matrix	89

1. Introduction:

In today's digital world, many people use the Internet for shopping, banking, studying, and social media. However, some websites are dangerous and can harm computers or steal personal information. These harmful websites include phishing sites that trick users into giving away passwords, malware sites that install viruses, and other cyber threats. Cybercriminals continuously develop new methods to deceive users, making it important to have advanced tools for protection. The increasing number of cyberattacks has highlighted the need for automated and efficient solutions to detect malicious websites before they cause harm. Traditional security methods like blacklists and manual analysis are often slow and ineffective against newly emerging threats.

To help people stay safe online we created a system that detects harmful websites using machine learning. Machine learning is a type of technology that allows computers to learn from data and make smart decisions. Our project studies different parts of a website's address URL to find out if it is safe or dangerous. Some of the things we check include URL length as very long URLs can be suspicious special characters since too many symbols like @ or - can be a sign of a fake website domain age, as websites that were created very recently may be risky and popularity since well-known websites are usually safer. We analyze content patterns, IP reputation, SSL certificate validity, hosting details, and traffic behavior to identify potential threats.

We use machine learning models like Random Forest and Support Vector Machines (SVM) to predict whether a website is safe, malicious, or suspicious. These models learn from past data and improve over time to give better results. Our project also includes a real-time tool where users can enter a website link and check if it is safe. This tool provides safety classifications and confidence scores to help users understand the risk level. To make the system even more accurate, we connect it with trusted sources like Google Safe Browsing and WHOIS(A network protocol and database that provides information about domain name and IP address). These services give extra information about the history and reputation of a website, enhancing the accuracy of our predictions.

The system is designed to continuously learn and adapt to new cyber threats by updating its database with the latest attack patterns. This ensures that the detection process remains effective even against newly developed malicious websites. By using machine learning, additional data sources, and continuous updates our project helps protect people from harmful websites. This system can be useful for both individuals and organizations, making the Internet a safer place for everyone. Future enhancements may include integration with browser extensions and mobile applications to provide real-time protection while browsing. With ongoing research and improvements, we aim to develop a highly robust system that significantly contributes to online safety.

2. Related Work (Existing System):

The current methods for detecting malicious URLs depend on various approaches, including machine learning and lexical feature-based analysis. These approaches have certain limitations that affect their accuracy and effectiveness in real-time environments.

1. Detecting Phishing URLs Using Machine Learning & Lexical Feature-Based Analysis:

- This method looks at the structure of URLs (like their length, special characters, subdomains, and certain words).
- Machine learning is used to teach a system to tell if a URL is safe or harmful based on these features.
- **Problem:** It can only detect known types of phishing URLs. If a URL is new and doesn't follow typical patterns, it might be missed.

2. Suspicious URL Filtering Using Logistic Regression with Multi-View Analysis:

- This method uses a model called **Logistic Regression** to classify URLs as safe or suspicious. It looks at things like the domain (website name), path (the link structure), and past data.
- **Multi-view analysis** means looking at URLs from different angles to improve accuracy.
- **Problem:** Logistic Regression has trouble when the data is complex or when phishing URLs evolve, so it might not catch more advanced threats.

3. Malicious URL Detection System Using Combined SVM and Logistic Regression Model:

- This method combines **Support Vector Machines (SVM)** and **Logistic Regression** for better accuracy.
- **SVM** is good at handling complex relationships between features, and **Logistic Regression** helps calculate the likelihood of a URL being malicious.
- **Problem:** It still faces challenges in detecting threats in real-time and struggles to validate data from external sources, making it less efficient for immediate detection.

Drawbacks of Existing System

1. **Only Detects Known Patterns:** These methods can only catch phishing URLs that follow common patterns. New types of phishing URLs might not be detected.
2. **Struggles with Advanced Phishing:** New and smart phishing tricks might not be caught by older methods, making them less effective.
3. **Too Many False Alarms:** Sometimes, safe websites are marked as dangerous, which can be annoying and confusing.
4. **Misses New Tricks:** Some methods only look for known warning signs in URLs, so new phishing techniques may slip through.
5. **Slow Detection:** Some methods take too long to check URLs, so they aren't good for detecting threats in real-time.
6. **Needs More Data:** If there isn't enough data for training, the model might not be able to catch all types of malicious URLs.
7. **Doesn't Check Other Sources:** Some methods don't check external sources (like security feeds), so they may miss new threats.
8. **Can't Handle Complex URLs:** Some methods, like Logistic Regression, have trouble understanding complicated patterns in URLs, which can reduce accuracy.
9. **Can Be Tricked:** Clever attackers can hide malicious parts of URLs, making it hard for the system to catch them.
10. **Doesn't Adapt Quickly:** These methods may not change fast enough to keep up with new phishing tricks, so they might become less effective over time.
11. **Limited to Basic Features:** These methods mostly focus on simple things like URL length or special characters and don't look deeply at the actual content of the website.
12. **Hard to Handle Big Data:** When there are too many URLs to check at once, these methods can slow down or struggle to process all of them quickly.

3. Problem Statement :

In the project "Cybersecurity Enhancement through Malicious URL Detection with Machine Learning," there are several key challenges to address. One of the biggest problems is that cyber threats, especially harmful websites, are becoming more advanced and harder to detect. Cybercriminals use clever tricks to hide these malicious websites, making it difficult for traditional security systems to catch them. This means that many of the older systems are not effective in protecting users from new and evolving threats.

Another issue is that existing systems often make mistakes when identifying harmful websites. Sometimes, safe websites are mistakenly flagged as dangerous, which can cause confusion for users. On the other hand, some harmful websites may go undetected, which poses a serious risk to internet security. This inconsistency in detecting malicious sites makes it harder for users to trust security systems.

Phishing websites, which pretend to be real websites to steal personal information, are another major challenge. These fake websites are designed to look very similar to legitimate ones, making it extremely difficult to tell them apart. Detecting phishing sites quickly is essential to protect users from identity theft and other types of cybercrime. However, since these phishing websites are often almost identical to the real ones, spotting them requires advanced technology.

To solve these challenges, machine learning can play a crucial role. Machine learning algorithms can analyze patterns in URLs and website behavior, helping to identify malicious sites more accurately. By training models with large amounts of data, these systems can become better at distinguishing between safe and harmful websites, reducing errors and improving trust in security systems.

Addressing these challenges is important for improving internet security and protecting users from the dangers of malicious websites. With better detection systems, we can prevent cybercrime and ensure a safer online experience for everyone.

4. Requirement Analysis, Risk Analysis, Feasibility Analysis

Requirement Analysis:

Requirement analysis is about understanding what this project needs to do. It includes:

1. Identifying **user needs**: People want to stay safe while using the internet. They need a tool that can check if a website is safe or dangerous.
2. Defining **features**: The system should analyse website links (URLs) and classify them as safe, suspicious, or harmful. It should give users clear reasons why a website is flagged.
3. Gathering **technical requirements**: The tool should use Machine Learning (ML) to detect dangerous websites. It must check URL length, special characters, domain age, numbers in the URL, and website popularity. It should also connect to security databases like Google Safe Browsing and WHOIS to get more information.
4. Understanding **constraints**: The tool must work fast, be easy to use, and support different devices (computers, phones, tablets).

Risk Analysis:

Risk analysis helps to find possible problems and prepare for them. Some risks include:

1. Technical Risks:
 - The machine learning model might misclassify URLs, marking a safe site as harmful or missing a dangerous one.
 - The system might face cyberattacks, like hackers trying to trick it.
2. Operational Risks:
 - The system might fail to update real-time data, making some website checks incorrect.
 - If too many people use the tool at the same time, it could slow down or crash.
3. External Risks:
 - The system depends on third-party services like Google Safe Browsing and WHOIS. If these services stop working, the accuracy of the tool may drop.
 - New cyber threats appear daily. If the system is not updated regularly, it may not detect the latest dangers.

Feasibility Analysis

Feasibility analysis helps determine whether a project can be completed. It looks at different factors to ensure the project is realistic and achievable.

1. Technical Feasibility

- This checks if the necessary technology is available and can be used for the project.
- For example, we need machine learning, security databases, and cloud hosting, which are all accessible and suitable for this project.

2. Economic Feasibility

- This ensures that the project is affordable and fits within the budget.
- It considers the cost of development, hosting, and maintenance to make sure it doesn't become too expensive.

3. Operational Feasibility

- This checks if the team has the right people with the necessary skills to complete the project.
- Skilled developers, testers, and cybersecurity experts are available to build and maintain the system.

4. Time Feasibility

- This ensures that the project can be completed within the planned schedule.
- It also considers the need for regular updates to keep the system safe from new cybersecurity threats.

5. Proposed System:

Detecting malicious URLs is crucial for protecting users from online threats like phishing, malware, and scams. Malicious URLs often appear normal but can redirect users to harmful websites that steal personal information or install viruses. To build a system that can automatically detect such URLs, we need a structured approach involving data collection, feature extraction, algorithm selection, model training, and evaluation.

The first step is collecting a dataset that includes both safe (benign) and harmful (malicious) URLs. A diverse dataset helps the model recognize different types of URLs. This dataset should be sourced from phishing databases, cybersecurity organizations, and legitimate web directories to ensure balanced learning. A well-prepared dataset allows the model to identify various URL patterns and better generalize them to new threats.

Once the dataset is collected, relevant features need to be extracted. Features are characteristics of a URL that help determine whether it is safe or malicious. Some important features include URL length, number of special characters, number of subdomains, presence of suspicious keywords, and whether the URL uses an IP address instead of a domain name. Malicious URLs often contain unusual patterns, such as excessive special characters or misleading keywords like "login," "secure," or "free." Extracting these features provides meaningful data for the machine learning model, allowing it to differentiate between safe and harmful URLs.

After feature extraction, selecting the right machine-learning algorithm is essential for accurate classification. Different algorithms process data in different ways. The naive Bayes Classifier is a simple yet effective model that works well when features are independent. Decision Trees create a set of rules to classify URLs based on feature values, while Random Forest improves accuracy by combining multiple decision trees. K-Nearest Neighbours (KNN) compares new URLs to known examples and classifies them based on their closest matches. Each algorithm has strengths and weaknesses, and testing multiple models can help identify the best-performing one for the given dataset.

Once an algorithm is selected, the model is built and trained using the dataset. Training involves feeding the model labelled data, allowing it to learn patterns in malicious and benign URLs. The more data it processes, the better it becomes at identifying threats. However, even a well-trained model must be evaluated to measure its accuracy and effectiveness. This is done by testing it on new, unseen URLs and using evaluation metrics like precision, recall, and F1-score. Precision checks how many detected malicious URLs are harmful, while recall measures how many real threats were correctly identified. The F1 score balances both precision and recall, providing an overall performance measure. A reliable model should correctly identify harmful URLs while minimizing false positives.

Building an effective system for detecting malicious URLs requires continuous improvement. Cyber threats constantly evolve, and new phishing techniques emerge regularly. To maintain accuracy, the model must be updated with new data and improved algorithms.

6. Advantages:

- 1. Protects from harmful websites** -Stops users from visiting fake or dangerous sites that steal data. It keeps personal information safe from hackers.
- 2. Saves time** -No need to check each link manually, as the system does it automatically. This helps users avoid risky websites quickly.
- 3. Works fast** -Detects and blocks malicious URLs in real-time. Users stay protected without delays.
- 4. Learns and improves** -Uses machine learning to get better at finding harmful links. Over time, it adapts to new cyber threats.
- 5. Can be used anywhere** - Works in browsers, emails, and security tools. Helps protect businesses and individuals from online threats.
- 6. Prevents money loss** - Stops phishing sites that try to steal bank details. Reduces the risk of fraud and financial scams.
- 7. Keeps users safe** - Protects passwords, credit card details, and private data. Ensures a secure browsing experience.
- 8. Updates regularly** - Can learn about new threats and improve detection. Keeps users protected from evolving cyberattacks

7. Literature Survey:

1. Enhancing cyber security by predicting malware using supervised machine learning models

The research paper talks about how machine learning can help find and stop malware which is harmful software that can attack computers. Malware can steal personal data damage files or cause financial loss. Traditional antivirus software looks for malware by comparing it to a list of known threats but hackers keep creating new malware every day that looks different. Because of this antivirus programs sometimes fail to detect new threats. To solve this problem the researchers used machine learning which can learn patterns from past malware and detect even new and unknown threats. They tested six machine learning models random forest k-nearest neighbours support vector machine decision tree naive bayes and logistic regression. Among this random forest worked the best with 99 percent accuracy meaning it almost always predicted correctly whether a file was safe or dangerous. The model checked file headers metadata and other details to decide if a file was harmful. Since it does not depend on old malware techniques it can even detect new types of malware.

The study shows that machine learning is very useful for cybersecurity because it can find malware more accurately than older methods. There are still some challenges. The model was tested on a pre-collected dataset but it needs to be tested on real-time malware to prove it works in daily use. Hackers may try to trick the model by changing how the malware looks. The researchers suggest that future work should test the model in real-world situations and improve its ability to detect advanced threats. They also recommend integrating machine learning models into antivirus software to make cybersecurity stronger. This research shows that machine learning can improve malware detection and make computers safer. With more testing and development, it could help people companies, and governments stay protected from cyber threats in the future.

- Dinesh Kalla Fnu Samaah and Dr. Sivaraju Kuraku.

2. Cyber Threat Intelligence-Based Malicious URL Detection Model Using Ensemble Learning

The research paper talks about a new way to find harmful websites using cyber threat intelligence and machine learning. Many websites on the internet can be dangerous because they have malware or trick people into giving personal information. Old methods of finding these websites do not work well because hackers change their tricks often. This study presents a new method that makes detection better by collecting information from web searches and security reports.

The researchers used three types of data. First, they looked at website URLs. Second, they collected WHOIS (A network protocol and database that provides information about domain name and IP address) information which gives details about website owners. Third, they gathered cyber threat intelligence from Google searches. They then trained machine learning models to check if a website is safe or harmful. The model used a two-step process. In the first step, the Random Forest algorithm sorted the data. In the second step, a Multilayer Perceptron made the final decision. The model was tested using twenty thousand website links and it showed a seven-point eight percent better accuracy and six-point seven percent fewer false alarms compared to older models.

This study shows that cyber threat intelligence is very useful in security. Older methods look only at the website content but hackers can change or hide that information. This new approach collects outside information to make a better decision. The researchers also tested different machine learning methods and they found that Random Forest combined with Multilayer Perceptron worked the best.

- Mohammed Alsaedi Fuad A. Ghaleb Faisal Saeed Jawad Ahmad and Mohammed Alasli

3. Machine Learning in Cybersecurity Techniques and Challenges

The research paper talks about how machine learning is used in cybersecurity to protect systems from online threats. Cyberattacks are increasing and traditional security methods are not always enough to stop hackers. This study explains different machine learning techniques that can help detect and prevent attacks more effectively.

The paper discusses various machine learning methods such as supervised learning unsupervised learning and reinforcement learning. Supervised learning is used to train models using past attack data so they can recognize future threats. Unsupervised learning finds hidden patterns in data without label examples helping discover unknown threats. Reinforcement learning improves security systems by learning from past actions and adjusting to new threats. These techniques help detect malware phishing attacks and suspicious network activities. The study also highlights challenges in using machine learning for cybersecurity. One big problem is getting enough quality data to train models. If the data is incorrect or biased the model may not work well. Another issue is that hackers can trick machine learning models by creating new types of attacks that the models do not recognize. Also, cyber threats keep changing so machine learning models need to be updated regularly to stay effective.

The research suggests improving cybersecurity with better data collection strong defense strategies and advanced machine learning models. Future work should focus on making models more adaptable to new threats and securing them from hacker manipulation.

- Jasmin Praful Bharadiya Et.al.

4. Machine Learning for Cybersecurity Detecting and Preventing Cyber Attacks.

The research paper talks about how machine learning can help in cybersecurity by finding and stopping cyber-attacks. Cyber-attacks are increasing and old security methods are not enough to stop them. Hackers use new tricks to steal data and damage systems. Machine learning can help by learning from past attacks and finding patterns to detect new threats.

The paper explains different machine learning methods. Supervised learning trains models using past data to recognize future attacks. Unsupervised learning finds hidden threats by looking for unusual activity. Reinforcement learning helps security systems improve by learning from past mistakes. These methods help detect malware phishing and suspicious network activities.

The study also discusses problems with using machine learning in cybersecurity. One problem is that models need a lot of good-quality data to work well. If the data is wrong the model may not detect attacks correctly. Another issue is that hackers can trick machine learning models by creating new types of attacks. Cyber threats keep changing so models need updates to stay useful.

The research suggests improving cybersecurity by collecting better data using strong security strategies and improving machine learning models. Future work should focus on making models stronger and harder for hackers to fool.

In conclusion, machine learning is a powerful tool for cybersecurity. It helps detect and stop cyber-attacks but it also has challenges. If improved it can protect computers networks and personal data from hackers more effectively.

- Prof. Agostino Marengo ,Dr. Alessandro Pagano Et.al.

5. Towards Detecting and Classifying Malicious URLs Using Deep Learning

The research paper talks about how machine learning can detect harmful websites by identifying malicious URLs. Cyberattacks often use fake links in emails and web pages to steal personal data or install harmful software. Traditional security methods like blacklists are not enough because new threats appear daily. This study uses machine learning to solve this problem.

The researchers tested different machine learning models including Random Forest Decision Tree k-nearest Neighbours and deep learning models. Using the ISCX-URL-2016 dataset they trained models to detect and classify harmful URLs like phishing and malware. Random Forest performed best with over 96% accuracy. The paper also discusses the challenges of using machine learning for cybersecurity. Hackers create new threats that models may not recognize. Machine learning requires high-quality data for accuracy and deep learning models take time and power to train.

The researchers suggest improving cybersecurity by making machine learning models faster and better at adapting to new threats. They also recommend selecting the most important features to improve accuracy and efficiency.

- Clayton Johnson Bishal Khadka Ram B. Basnet and Tenzin Doleck.

6. Malicious URL Detection based on Machine Learning

The paper Malicious URL Detection Using Machine Learning talks about how machine learning can help find dangerous websites on the internet. As more people go online the risk of visiting harmful websites that steal personal data or spread viruses is increasing. The authors wanted to find a way for computers to automatically detect these harmful websites.

Every website has a unique web address (URL) and some patterns in URLs can indicate whether a site is safe or dangerous. These patterns include URL length special symbols and domain type. By studying these patterns machine learning can teach computers how to identify unsafe websites.

The authors used machine learning algorithms which are like smart programs that learn from examples. They trained the system with both safe and harmful URLs so it could recognize dangerous websites on its own. After learning the system could classify URLs automatically without needing human help.

The paper also compares different machine-learning methods and explains how they can be used in real life. Traditional security systems use fixed rules to find harmful sites but machine learning is faster and more accurate because it adapts and improves over time.

In the end, the paper shows that machine learning is an effective way to make the internet safer by blocking harmful websites. This method can be used in web browsers or security software to protect users from online threats.

-- Cho Do Xuan Et.al.

7. Malicious URL Detection Using Machine Learning

The paper Malicious URL Detection Using Machine Learning talks about how computers can help find dangerous websites on the internet. Since more people are using the internet the risk of visiting harmful websites has also increased. Some websites steal personal details or install viruses so the authors wanted to find a way to automatically detect these harmful sites using computers.

Every website has a URL (web address) and some patterns in URLs can show if a website is safe or not. These patterns include URL length special symbols and domain type. The idea is to study these patterns and use machine learning to train a computer to tell safe and harmful websites apart.

The authors used machine learning algorithms which are like smart programs that learn from examples. They trained the system using both safe and harmful URLs so the computer could recognize the differences. Once trained the computer could classify URLs automatically without needing a person to check them.

The paper also compares different machine-learning methods and explains how they work in real-life situations. Traditional security methods use fixed rules to detect threats but machine learning is faster and more accurate because it keeps learning and improving over time.

In the end, the paper shows that machine learning is a powerful tool for making the internet safer. This method can be added to web browsers or security software to protect people from harmful websites.

-- Ferhat Ozgur Catak Et.al.

8. Detecting Malicious URLs Using Machine Learning Techniques Review and Research Directions

This paper Detecting Malicious URLs Using Machine Learning Techniques Review and Research Directions explains how machine learning helps find dangerous websites. Since more people use the internet cybercriminals create fake websites to steal personal data or spread viruses. To stop this researchers have developed machine learning models that can identify harmful websites.

The authors looked at many studies on how machine learning helps detect malicious URLs. They explain that every website has a unique address (URL) and some patterns in the URL can tell if a site is safe or harmful. For example, a very long URL random characters, or unusual words can be signs of a dangerous website. Machine learning models study these patterns and learn to separate safe websites from harmful ones.

The paper also discusses different machine-learning techniques used for this task. Some models follow simple rules while others analyse large datasets to improve accuracy. The authors compare different models to see which ones work best. They also talk about the datasets researchers used to train machine learning systems.

Even though machine learning is helpful the paper highlights some challenges. One problem is that cybercriminals keep changing their tricks making it hard for models to keep up. Another issue is that sometimes safe websites are wrongly flagged as dangerous which can be frustrating. The paper suggests improving training data and combining multiple detection methods to solve these issues.

In the end, the paper shows that machine learning is a powerful tool for finding harmful websites. It helps improve cybersecurity and protect users online. However, better models are needed to handle new threats. The study encourages further research to make the Internet even safer.

--Malak Aljabri Et.al.

9. Malicious URL Detection Based on Machine Learning

The paper Malicious URL Detection Based on Machine Learning explains how smart computer programs can help find dangerous websites. These websites can steal personal data spread viruses or trick people into sharing passwords. Since the internet is full of such threats detecting them quickly is important.

The authors explain that every website has a unique address (URL) and some URLs have hidden signs that show if a site is safe or harmful. These signs include weird words extra-long links random letters or strange symbols. Checking websites manually is difficult so machine learning is used to teach computers how to spot dangerous links automatically.

To do this the researchers collected many URLs some safe and some harmful and used them to train a machine-learning model. The model studies URL patterns and learns to separate safe sites from harmful ones. Once trained the computer can quickly check new URLs and predict whether they are dangerous or not.

The paper also explores different machine-learning techniques for this task. Some methods follow simple rules while others analyze huge amounts of data to improve accuracy. The authors compare different models to find which one works best.

However, there are challenges too. Hackers keep changing their tricks so new types of dangerous URLs keep appearing. Also, sometimes safe websites are wrongly marked as harmful which can be frustrating. To solve this the authors suggest improving training data and enhancing how models learn from new threats.

In the end, the paper shows that machine learning is a powerful tool for detecting harmful websites. If used properly it can help protect users from online scams and cyberattacks.

-- Dr. J. Sarada Et.al.

10. Malicious URL Classification Using Artificial Fish Swarm Optimization and Deep Learning

The paper Malicious URL Classification Using Artificial Fish Swarm Optimization and Deep Learning explains how smart computer programs can help detect dangerous websites. Many websites try to steal personal data spread viruses or trick people so finding them quickly is important for online safety.

The authors use Deep Learning along with a unique method called Artificial Fish Swarm Optimization (AFSO) to improve how computers identify harmful websites. AFSO is inspired by how fish move in groups to find food and in a similar way it helps computers learn patterns in URLs better and make more accurate predictions.

The research explains that every website has a unique address (URL) and some URLs have hidden clues that show whether they are safe or dangerous. These clues include random letters extra-long links or unusual symbols. Since checking each website manually is impossible machine learning and deep learning train computers to recognize harmful URLs automatically.

To do this the authors collected many safe and harmful URLs and used them to train a deep-learning model. This model studies URL patterns and learns how to separate good websites from bad ones. The AFSO technique makes the model even better by helping it find the best features for classification making detection more efficient.

The paper also discusses challenges in detecting malicious URLs. Hackers constantly change their tactics so the model needs regular updates. Another issue is that some safe websites might be wrongly marked as dangerous which can be frustrating for users.

Overall the study shows that using Deep Learning and AFSO together makes malicious URL detection more accurate and reliable. This research helps improve cybersecurity and protect people from online threats. The authors suggest that future improvements can make the system even smarter and stronger.

--Anwer Mustafa Hila Et.al.

11. Detecting Malicious URLs Using Machine Learning

The paper Detecting Malicious URLs Using Machine Learning talks about how the internet is full of threats and how cybercriminals often use malicious URLs (links) to attack users by tricking them into downloading viruses visiting fake websites or revealing personal information. Because of this detecting harmful URLs is essential for online security. Traditional security methods such as firewalls and blacklists (lists of known dangerous websites) have been used to protect users but they have limitations. Firewalls can slow down devices and blacklists are ineffective against new or unknown threats. To overcome these challenges researchers have turned to machine learning (ML) and deep learning to improve detection techniques.

Instead of relying on fixed rules ML models analyze URL patterns and behaviors to determine if a link is safe or harmful. In this study, researchers used two machine learning techniques Support Vector Machine (SVM) and Random Forest (RF). Additionally, they proposed a new approach to extract useful features from URLs incorporating both static (unchanging) and dynamic (behavior-based) characteristics. This method enhances the accuracy of malicious URL detection.

To further improve results researchers tested a more advanced deep learning model called Deep Bidirectional Long Short-Term Memory (DBLSTM). This model processes URLs as a sequence of time-based data allowing it to detect more complex threats that traditional methods might miss. The results showed that deep learning models performed better than traditional machine learning techniques. The researchers also made their code available on GitHub enabling others to use and refine their work. In the future, they plan to test their model against real-time cyberattacks and further enhance its accuracy.

By leveraging AI and deep learning detecting malicious URLs becomes more efficient and automated. This research contributes to improved cybersecurity helping protect networks personal data and critical online systems from potential cyber threats.

-- Mr.K.V.V.Subbarao Et.al.

12. Malicious URL Detection Using Machine Learning

The paper Malicious URLs Detection Using Machine Learning talks about how Malicious URLs are links that lead users to harmful or fraudulent websites posing serious cybersecurity risks. These links are often used for phishing spamming and spreading malware which can result in financial losses and data breaches. The average cost of a data breach is estimated to be \$4.24 million making it crucial to detect and prevent these threats. Traditionally blacklists have been used to block harmful URLs but they are not always effective against newly created threats. To improve detection machine learning techniques such as Random Forest and Support Vector Machines (SVM) have been introduced offering better accuracy and reliability. This research provides an overview of machine learning methods for identifying malicious URLs and explores more effective approaches for cybersecurity.

With the growth of e-commerce and online banking, many websites collect user data for authentication. However, distinguishing between safe and malicious sites is challenging. Cybercriminals take advantage of this by using fake websites to steal personal and financial information. Malicious URLs are designed to deceive users and spread cyber threats. Since malware and hacking techniques constantly evolve traditional detection methods often fail. As a result machine learning models are increasingly used to analyze URL patterns and identify potential threats.

Recent studies highlight the rising number of phishing attacks with reports showing that phishing sites outnumber malware-hosting sites by 75 times. Webmail and SaaS platforms are common targets accounting for 34.7% of phishing attempts. Business Email Compromise (BEC) scams are also increasing with Gmail being the most targeted email provider. Around 25% of all data breaches result from phishing and social engineering attacks. Research has explored various machine learning models particularly Random Forest and SVM to detect malicious URLs based on accuracy and computational efficiency.

The detection process begins with collecting and cleaning datasets that contain both safe and harmful URLs. Features like URL length and server type are analyzed. Machine learning models such as logistic regression decision trees SVM and neural networks are trained and evaluated using accuracy and precision metrics. Additionally, quantum machine learning (QML) is being explored as a new approach using quantum computing technology.

Python along with libraries like Pandas NumPy and Scikit-learn is commonly used for data processing. Quantum computing tools like IBM Quantum and Google Quantum AI provide advanced processing capabilities. This research emphasizes the effectiveness of machine learning in detecting cyber threats. Future work should focus on improving detection techniques and exploring hybrid models to enhance cybersecurity further.

-- Sanika Ambadkar Et.al.

13. Malicious URLs Detection

The paper Malicious URLs Detection talks about the increasing use of online services cyber threats like phishing and malware attacks have also grown. Hackers use malicious URLs to trick users into revealing personal information or downloading harmful software. Traditional methods such as blacklists struggle to keep up with new threats. As a result, researchers have turned to machine learning techniques to improve malicious URL detection.

This study focuses on using machine learning to identify harmful URLs based on their structure and behavior. Instead of relying on static blacklists, our approach analyses features such as URL length the number of special characters, and domain-related attributes. The dataset includes both safe and malicious URLs which are used to train and test machine learning models. Techniques like logistic regression Naïve Bayes and Random Forest were evaluated to find the most accurate and efficient model. Among these Random Forest performed the best providing reliable results in detecting threats.

To enhance detection, we also explored techniques like fuzzy string matching which helps identify URLs designed to mimic legitimate websites. Additionally, we used feature extraction methods such as TF-IDF (Term Frequency-Inverse Document Frequency) to analyze text patterns in URLs. These methods improve the accuracy of detecting deceptive URLs.

Our research highlights the importance of real-time detection as cyber threats continuously evolve. Future work should focus on improving machine learning models by integrating real-time data website content analysis and network behavior monitoring. Ethical concerns such as privacy and bias in machine learning models should also be addressed. Another challenge is adversarial attacks where cybercriminals manipulate URLs to bypass detection. Strengthening models against such threats is essential.

In conclusion, machine learning offers a promising solution for detecting malicious URLs. By analyzing their features and behaviors these models can significantly improve cybersecurity. Ongoing research should aim to refine these techniques and develop more robust real-time solutions to combat evolving cyber threats.

--Rekha Baghel Et.al.

14. Malicious URL Detection Using Machine Learning MASK

The paper Malicious URLs Detection Using Machine Learning MASK talks about how the internet has become an increasingly dangerous place with cyber threats rising every year. In 2011 alone cyber-attacks increased by 36% leading to approximately 4 500 new attacks per day. Traditional security tools struggle to keep up with these threats, especially with the widespread use of smartphones and mobile internet. This has given attackers a larger pool of potential victims making web security more important than ever. One of the most common cyber threats involves malicious URLs which are designed to steal personal data spread malware or deceive users into scams. These attacks often involve fake websites that appear legitimate tricking users into entering sensitive information. While security teams have implemented blacklists—databases of known malicious sites—these are not always effective at detecting new threats as cybercriminals constantly change their tactics. To combat this issue our study focuses on using machine learning to detect harmful URLs. By analyzing various features of URLs such as their structure and popularity machine learning models can identify potential threats more efficiently. Malicious URLs often have certain characteristics including excessive length suspicious keywords and untrustworthy hosting. To build our detection system we gathered a dataset containing both safe and malicious URLs. Using machine learning techniques, we trained a model to recognize patterns in harmful URLs and predict whether a new URL is dangerous. Specifically, we implemented the Random Forest algorithm which combines multiple decision trees to enhance accuracy. This process involved several steps including data preprocessing training the model testing its accuracy and evaluating the results using a confusion matrix—a tool that measures correct and incorrect predictions. Our model achieved 81% accuracy and an F1 score of 74% demonstrating its effectiveness in detecting malicious URLs. However, cybercriminals continuously evolve their strategies making it necessary to improve detection techniques. Future research should focus on deep learning methods more advanced feature extraction and real-time learning systems to enhance accuracy and adaptability. Although significant progress has been made in malicious URL detection it remains a challenging problem. Security tools must keep evolving to counter emerging cyber threats and artificial intelligence advancements will play a crucial role in making web browsing safer while preventing online fraud and cybercrime.

--Mayur Murarka Et.al.

15. Detection of Malicious URLs using Machine Learning based on Lexical Features

This paper presents a machine learning-based approach for detecting malicious URLs using lexical features. Traditional methods like blacklisting and heuristic classification struggle to keep up with evolving cyber threats. Hence, this study focuses on 23 lexical features, such as domain names, path lengths, and special characters, to classify URLs as benign or malicious. An Exploratory Data Analysis (EDA) is conducted to refine feature selection, improving classification accuracy.

The study compares three classifiers: Random Forest (RF), a bagging technique, and XGBoost and LightGBM, both boosting techniques. The dataset, sourced from a Malicious URL dataset, contains 651,191 URLs. The data is split into 80% training and 20% testing. The classification process involves feature extraction, training, and evaluation using accuracy, precision, recall, and F1-score.

Experimental results show that RF outperforms the other models with 96.6% accuracy, followed by LightGBM (95.6%) and XGBoost (93.2%). While boosting techniques typically improve performance, RF remains the most effective for this task. Notably, hostname length, number of directories, and first directory length are among the most influential lexical features.

Compared to previous works, this study advances malicious URL detection by using a refined feature selection process and advanced boosting methods. Prior studies, such as those by Mamun et al. and Apoorva et al., achieved accuracy rates of 97% and 92%, respectively. However, their models did not employ an extensive EDA process or advanced boosting techniques like LightGBM.

In conclusion, the study demonstrates that lexical features alone can effectively detect malicious URLs. RF remains the best-performing model, though future research aims to optimize LightGBM parameters to potentially surpass RF's accuracy. The findings have practical applications in cybersecurity, browser security, and phishing prevention.

-Prabodha Abeynayake Et.al.

16. Machine Learning for Malicious URL Detection

This paper explores the prevalent cybersecurity threat in which attackers evade security defenses and exploit URLs to carry out various malicious attacks on unsuspecting individuals. This research paper proposes a MuD (Malicious URL Detection) model that uses three supervised machine learning classifiers support vector machine (SVM), logistic regression, and Naive Bayes. For any URL, 15 features are selected and categorized as either lexical, host-based, or content-based. This model describes a process of detecting malicious URLs using three different machine learning algorithms and evaluates which algorithm works best. 5 steps were taken to build and train the model effectively and efficiently to detect malicious URLs. The steps are data gathering, data preprocessing, classification, training and testing, and evaluation. Web attacks are usually performed using compromised URLs that are malicious. A pre-processed dataset from the UCI Machine Learning Repository was used containing 11,055 URLs (both malicious and benign) with 30 extracted features. Some of the features used are having an IP address, URL length, having @ symbol, double slash, link in tags, having a sub-domain, age of domain, DNS record, HTTPS token, page-rank, google index, iframe, redirect, pop-up window and favicon. This model is tested on three machine learning classifiers and the accuracy, recall, and precision of each classifier is recorded. In classifying URLs SVM ran slower as compared to Navie Bayes and Logistic regression. Using lexical, host-based, and content-based features played an important role in their classifiers. The prediction model was tested using logistic regression, achieving an accuracy of 96%. Several malicious and benign URLs from PhishTank were used to evaluate its classification performance.

--Gold Wejinya and Sajal Bhatia Et.al

17. Enhancing Malicious URL Detection: A Novel Framework Leveraging Priority Coefficient and Feature Evaluation

The paper addresses the significant cybersecurity threat posed by malicious URLs, which are commonly used in phishing and malware distribution attacks. The authors propose an innovative framework that enhances detection capabilities by assigning priority coefficients to predefined static features of URLs. This approach aims to improve the accuracy and efficiency of identifying malicious URLs by evaluating and prioritizing specific characteristics that are indicative of malicious intent. To address these challenges, this study introduces a novel priority coefficient-based feature evaluation framework to enhance malicious URL detection. The framework incorporates 42 feature classes categorized into blacklist, lexical, host-based, and content-based features. The methodology involves the use of a dataset consisting of 5000 real-world URLs collected from phishing and malware sources such as URLhaus and PhishTank. The framework's performance is evaluated using machine learning models, including Support Vector Machine (SVM), Random Forest (RF), and Bayesian Network (BN). Benchmarking results demonstrate that the proposed framework outperforms existing methods achieving an impressive accuracy of 98.95% and a precision value of 98.60%. This research introduces several key contributions. The framework enables heuristic-based real-time detection, improving the identification of emerging threats. It utilizes predefined static feature classification, ensuring a structured approach to feature extraction. The priority coefficient mechanism assigns weight to critical feature classes, enhancing detection precision. Additionally, the feature evaluation process ensures robust decision-making, even when some features lack data. In conclusion, the proposed framework significantly enhances the accuracy of malicious URL detection and strengthens cybersecurity defenses against evolving threats. Future research will focus on further improving real-time detection capabilities and adapting the framework to counteract emerging cyber threats.

*-- Ahmad Sahban, Norshaliza Binti,
Mehran Behjati, Saad Aslam, Aaliya Sarfaraz and Angela Amphawan.*

18. Safeguarding cyberspace Enhancing malicious website detection with PSO–optimized XGBoost and firefly-based feature selection

The study introduces a two-step machine-learning model designed to detect malicious websites. It employs a Firefly-based feature selection technique to identify the most relevant website features, followed by an optimized XGBoost algorithm, which is fine-tuned using Particle Swarm Optimization (PSO) for improved classification performance. The model was evaluated using a dataset of over 36,000 websites. It achieved an accuracy of 98.42% in binary classification and over 98% accuracy across multiple classes. When compared to other classification models, including J48, Random Tree, Naive Bayes, and MLP, the PSO-XGBoost model outperformed them in terms of accuracy and F1 score. The study also highlights the importance of feature selection in enhancing detection performance. A total of 23 key features were identified as significant in distinguishing malicious websites. Among these, URL length, the number of dots in the domain, and character composition were found to be particularly influential. This approach offers several advantages, including higher detection accuracy with fewer false positives, reduced computational complexity due to efficient feature selection, and an adaptive, scalable model that can be further optimized to counter evolving cyber threats. The research concludes that the PSO-XGBoost model is highly effective for malicious website detection. Future work should focus on expanding the dataset with real-world data and integrating the model into real-time cybersecurity systems. Additionally, the study underscores the potential of swarm intelligence techniques in advancing cybersecurity solutions

-- Saeid Sheikhi, Panos Kostako *Et.al.*

19. Classification of Malicious URLs Using Machine Learning

Cybersecurity plays a vital role in safeguarding users from cyberattacks, particularly through malicious URLs. Cybercriminals use phishing, defacement, and malware techniques to exploit user vulnerabilities. Traditional blacklist-based methods for detecting malicious URLs are limited due to the emergence of new threats. Machine learning provides an effective alternative, with classification models enabling the identification of malicious URLs. However, training on large datasets can be computationally expensive, necessitating instance selection methods. A dataset of 650,000 URLs categorized as phishing, defacement, malware, or benign was collected from Kaggle. Feature extraction was performed, resulting in 16 input features, including URL length, domain length, presence of IPv4, HTTPS/HTTP, special character counts, and keyword presence. The dataset was split into 85% training (552,500 instances) and 15% testing (97,500 instances). The instance selection methods are DRLSH: which identifies and removes redundant samples with each class to improve efficiency, BPLSH: Which considers both similar and border instances to refine dataset representation and Random selection: which selects a random subset of data, offering simplicity and low computational cost. The machine models used are Decision trees, Random forests, Support Vector Machines, and K-nearest neighbors Model performance was evaluated on the test set using accuracy, precision, recall, and F1-score. The application of instance selection methods significantly reduced training time while maintaining classification performance. Among the models, RF demonstrated the highest accuracy, followed by SVM, DT, and KNN. Machine learning offers a powerful approach to detecting malicious URLs. Instance selection methods enhance computational efficiency without sacrificing accuracy. Future research should explore deep learning techniques and real-time detection mechanisms to further improve cybersecurity defenses.

--Shayan abad, Hassan gholamy
and Mohammad Aslani. *Et.al.*

20. Malicious URL Detection Using Machine Learning: A Survey

This paper analyses that Malicious URLs, also known as malicious websites, pose significant cybersecurity threats by hosting unsolicited content such as spam, phishing attempts, and drive-by downloads. Traditionally, malicious URL detection relied on blacklists—databases of known malicious URLs compiled over time. Although effective in identifying previously flagged URLs, blacklists have several limitations. They cannot detect newly generated malicious URLs and are susceptible to obfuscation techniques used by attackers. These obfuscation methods include modifying hostnames, using URL shortening services, and altering domain names to appear legitimate. Additionally, attackers often use fast-flux techniques, algorithmically generated URLs, and simultaneous multi-attack strategies to evade detection. Given these limitations, researchers have increasingly turned to machine learning (ML) techniques for malicious URL detection. Unlike blacklists, ML-based approaches can generalize to detect new threats by learning patterns from historical data. This capability makes them significantly more robust and adaptable to evolving cybersecurity threats. The problem foundation for malicious URL detection using machine learning is typically formulated as a binary classification problem, where URLs are categorized as either malicious (1) or benign (-1). The key challenge in this process is Feature Representation Converting a URL into a structured set of features that can be analysed by machine learning models. Machine Learning Model Development Training is a predictive function that accurately classifies URLs based on extracted features. Feature representation for malicious URL detection includes lexical features, Host-based features, Network-based features, Content-based features, and behavioral features. Machine learning approaches for malicious URL detection are naive Bayes, support vector machines, logistic regression, detection trees, random forests, and neural networks.

--Doyen Sahoo, Chenghao LIU, Steven *Et.al.*

8. Architecture Design:

Class Diagram:

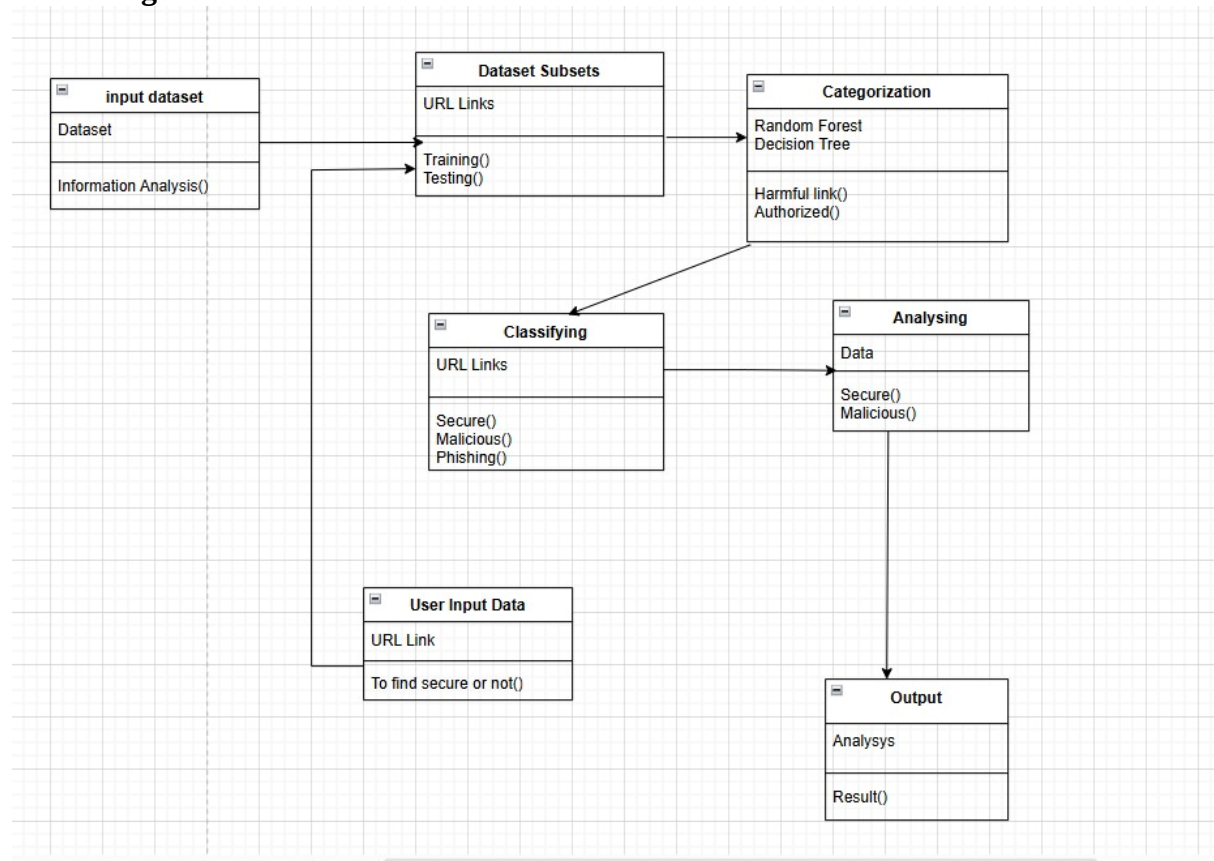


Figure-8.1 Class Diagram of URL Guardian

Sequence Diagram:

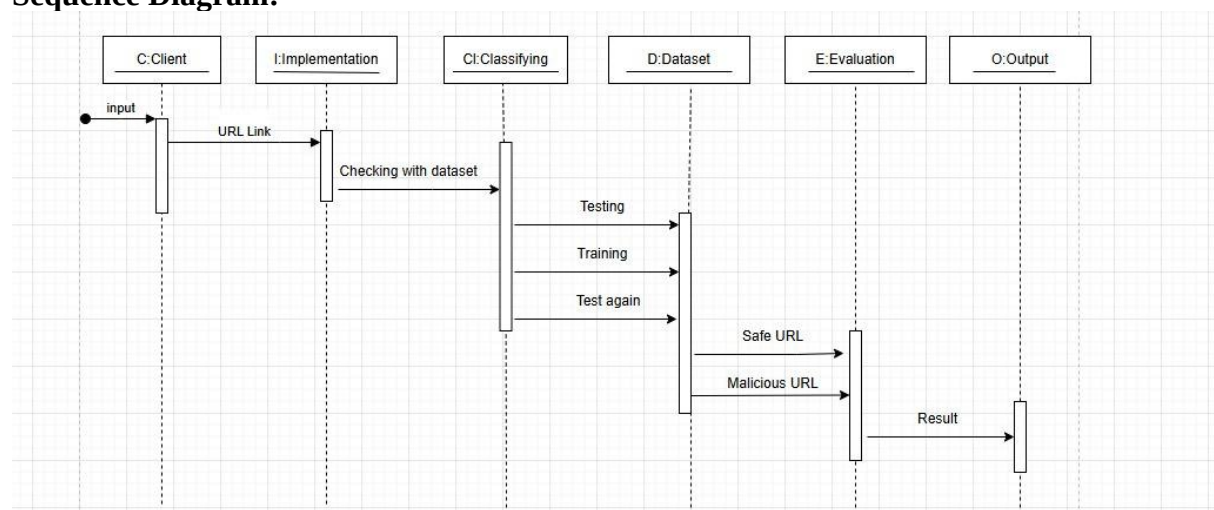


Figure-8.2 Sequence Diagram of URL Guardian

Use-Case Diagram:

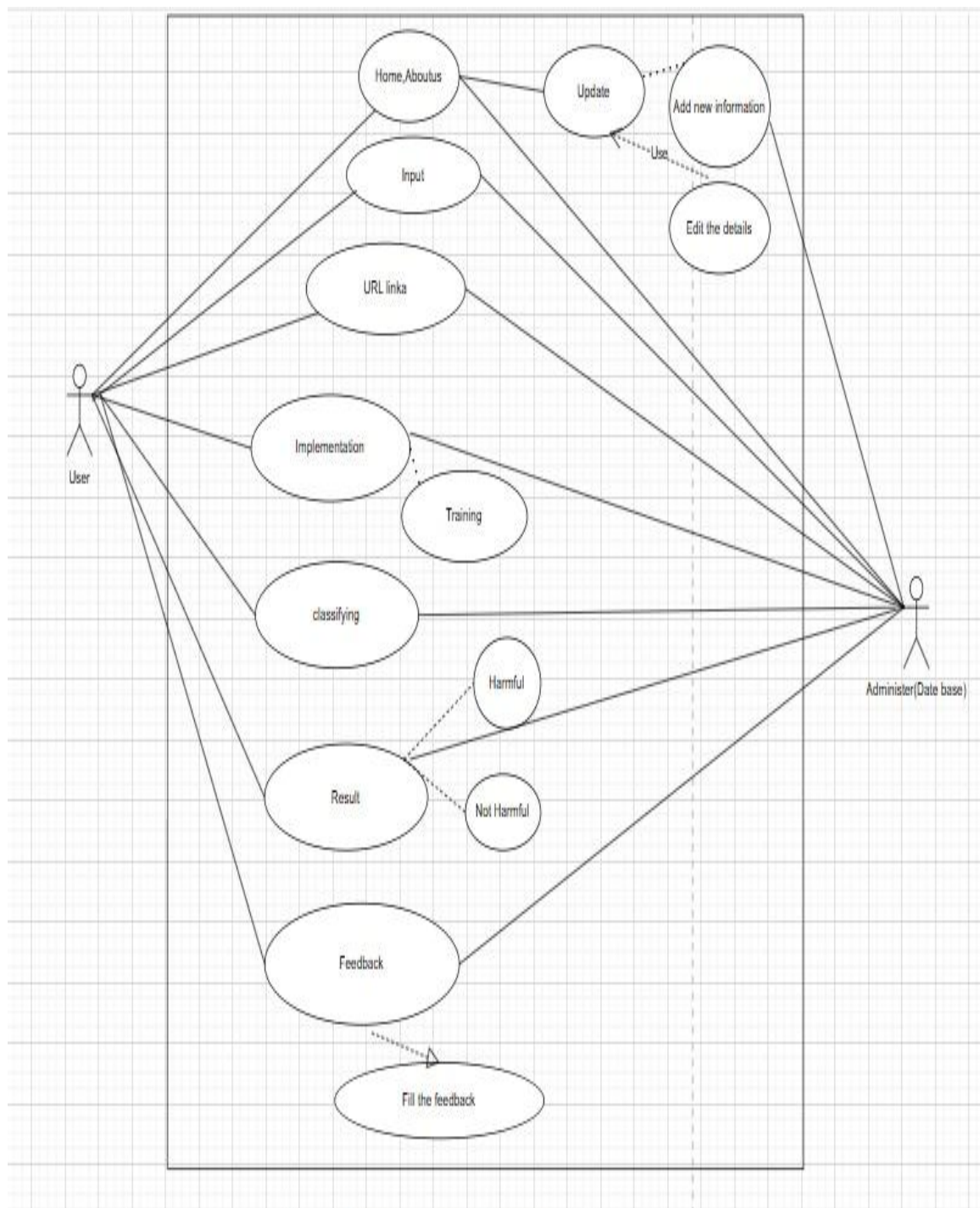


Figure-8.3 Use-Case Diagram of URL Guardian

Activity Diagram:

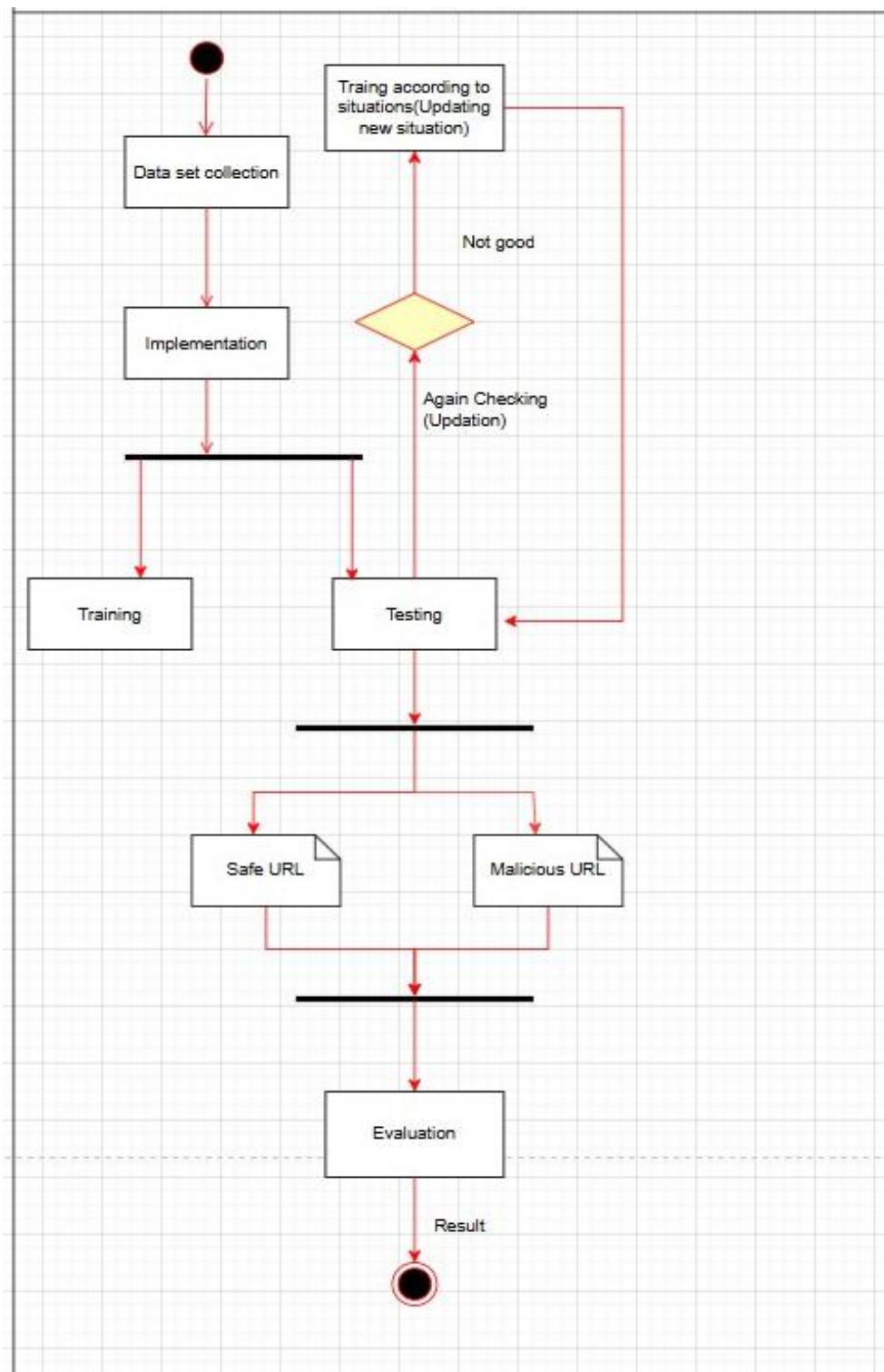


Figure-8.4 Activity Diagram of URL Guardian

Data Flow Diagram:

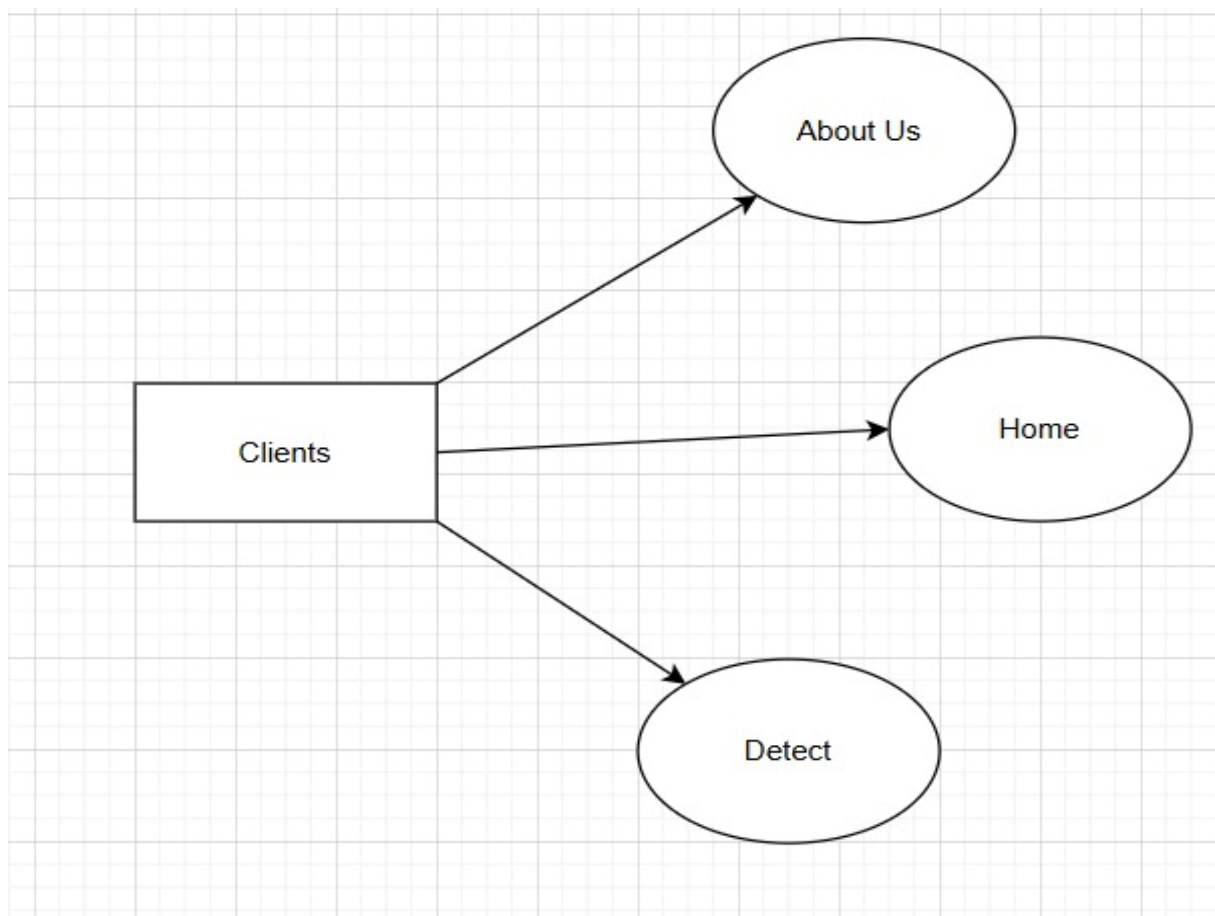


Figure-8.5 Data Flow Diagram Level-0

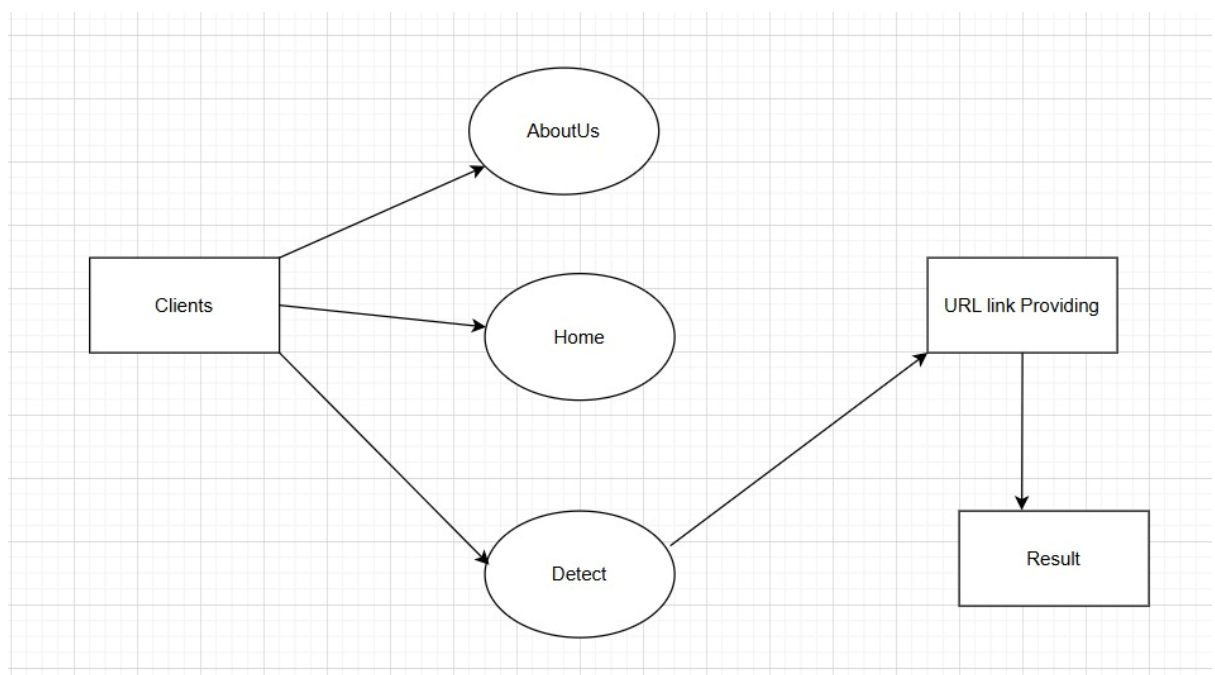


Figure-8.6 Data Flow Diagram Level-1

Architecture Diagram:

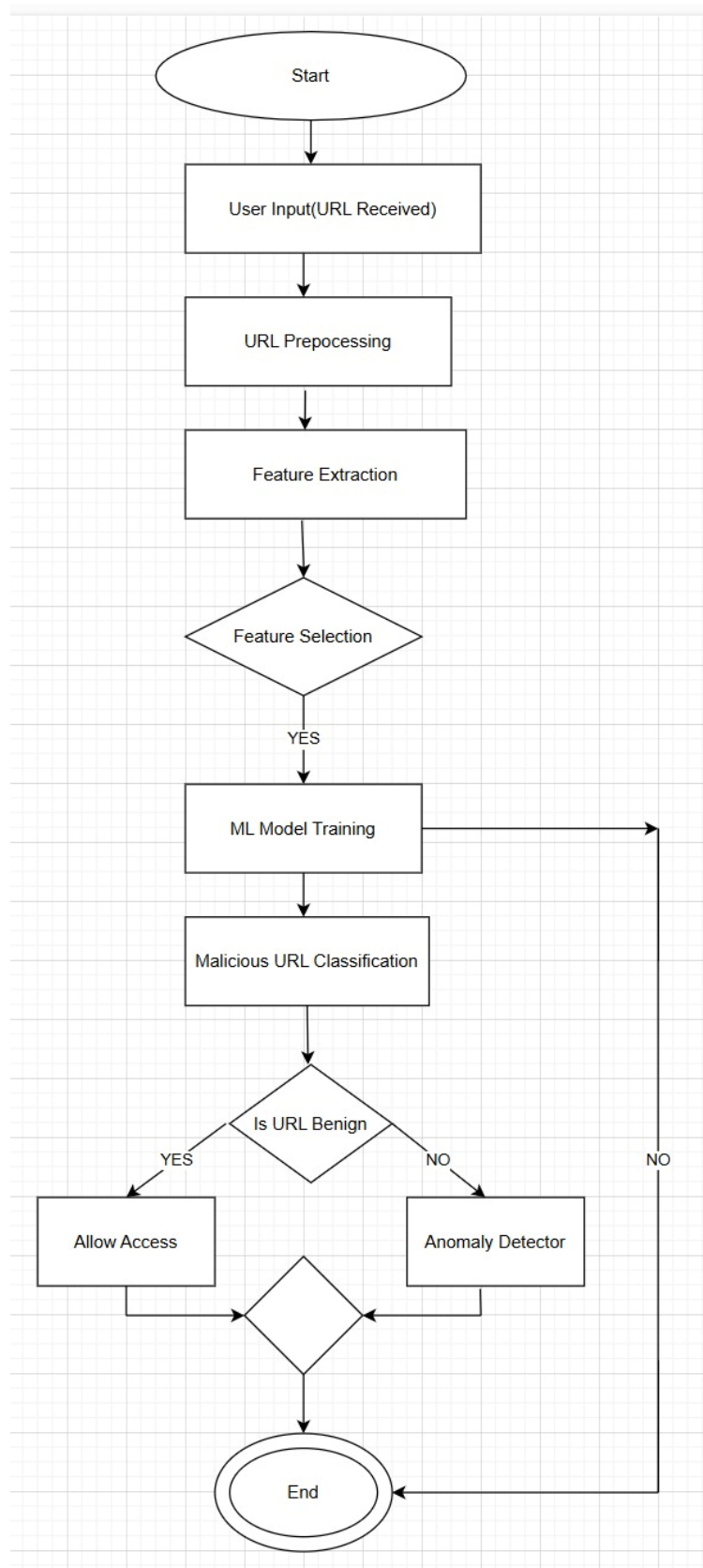


Figure-8.7 Architecture Diagram of URL Guardian

9. Implementation

1. Gathering Dataset

- Collect benign (safe) and malicious URLs from various sources.
- Use databases like Phish Tank, Google Safe Browsing, and WHOIS to label data.
- Extract key features like:
 - URL Length
 - Number of special characters
 - Presence of numeric values
 - Domain age
 - SSL certificate validity
 - Hosting details & redirection patterns

2. Training

- Use Supervised Machine Learning to train models.
- Apply algorithms such as Random Forest and Support Vector Machines (SVM).
- Train the model on labeled datasets to distinguish between safe and harmful URLs.
- Integrate external security databases (e.g., Google Safe Browsing) to improve accuracy.

3. Testing

- Validate the model using test datasets.
- Measure Precision, Recall, and F1-score to check accuracy.
- Ensure the system correctly flags malicious URLs and minimizes false positives.

4. Asking User to Login

- Implement a login system for users.
- Store login credentials securely using encryption.
- Allow users to check URLs through an interface after authentication.

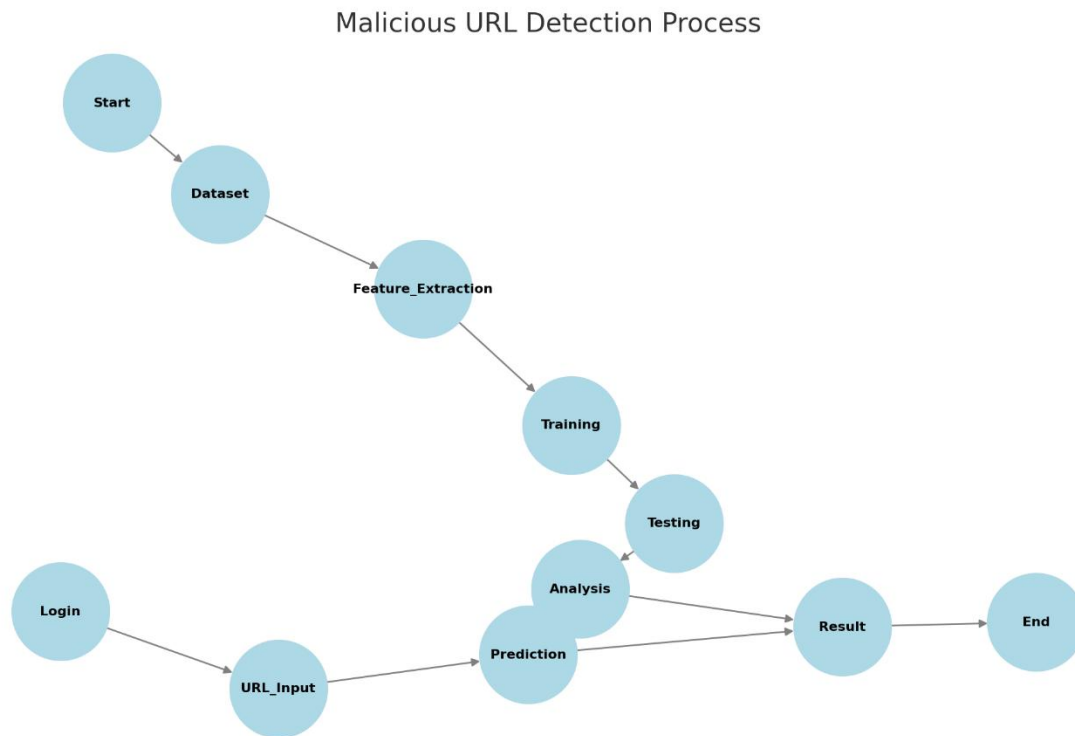


Figure-9.1 Malicious URL Detection Process

5. URL Link

- Provide a web-based interface or API where users can input a URL.
- The system returns results: Safe, Malicious, or Suspicious, with a confidence score.
- Optionally, integrate with Google Safe Browsing API for additional security checks.

6. Testing

- Conduct real-time testing with unknown URLs.
- Compare detection accuracy with existing security solutions.
- Optimize performance for speed and reliability.

7. Algorithms Used

- Random Forest: A decision-tree-based ensemble method that provides high accuracy.
- Support Vector Machine (SVM): Efficient in detecting complex patterns in URL structures.
- Logistic Regression & Naïve Bayes: Used for probability-based classification.
- Deep Learning (Optional): Neural networks for advanced feature learning.

- Only Detects Known Patterns

The system can only find harmful links that look like old phishing links. New tricks may not be detected. The solution is to use AI models that can learn and detect new types of harmful links.

- Struggles with Advanced Phishing

Hackers create smarter fake websites that older systems cannot catch. The solution is to use a combination of different models such as Random Forests, Support Vector Machines, and Neural Networks to improve detection.

- Too Many False Alarms

Safe websites are sometimes wrongly marked as dangerous. The solution is to check links using multiple methods such as URL structure, SSL certificate, and domain reputation before marking them as harmful.

- Misses New Tricks

Some systems only look for old signs of phishing and may miss new techniques. The solution is to keep updating the system with new phishing examples from trusted sources like Google Safe Browsing.

- Slow Detection

Some systems take too long to check if a link is safe. The solution is to use faster models that work in real-time and can process multiple links at once.

- Needs More Data

If there are not enough examples of bad links in the training data, the system may not work well. The solution is to use a larger dataset from sources like PhishTank or OpenPhish to improve accuracy.

- Does Not Check Other Sources

Some tools do not verify information with security databases. The solution is to connect with security APIs like Google Safe Browsing to check if a link is dangerous.

- Cannot Handle Complex URLs

Some tools cannot understand tricky URLs that hackers create. The solution is to use AI-powered tools that analyze deeper patterns in URLs.

- Can Be Tricked

Hackers may hide harmful parts of links to avoid detection. The solution is to decode hidden parts of links and analyze them properly.

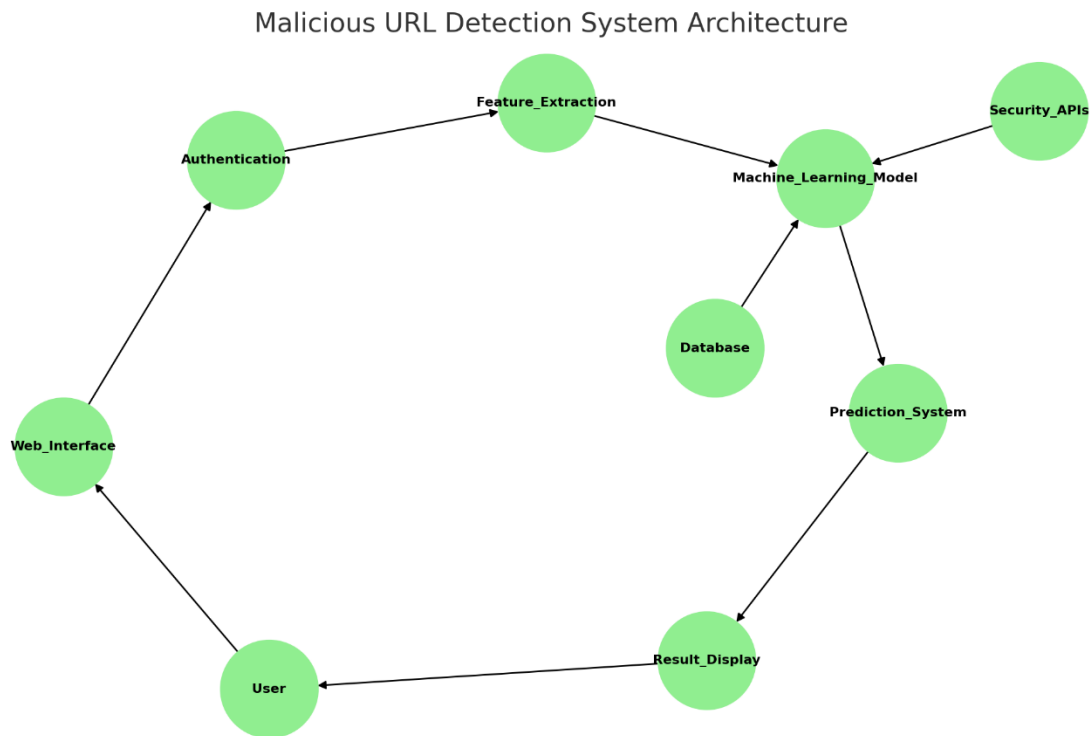


Figure-9.2 Malicious URL Detection System Architecture

8. Analysis

After training and testing the model, an analysis phase is necessary to evaluate performance and refine the system.

Feature Importance Analysis

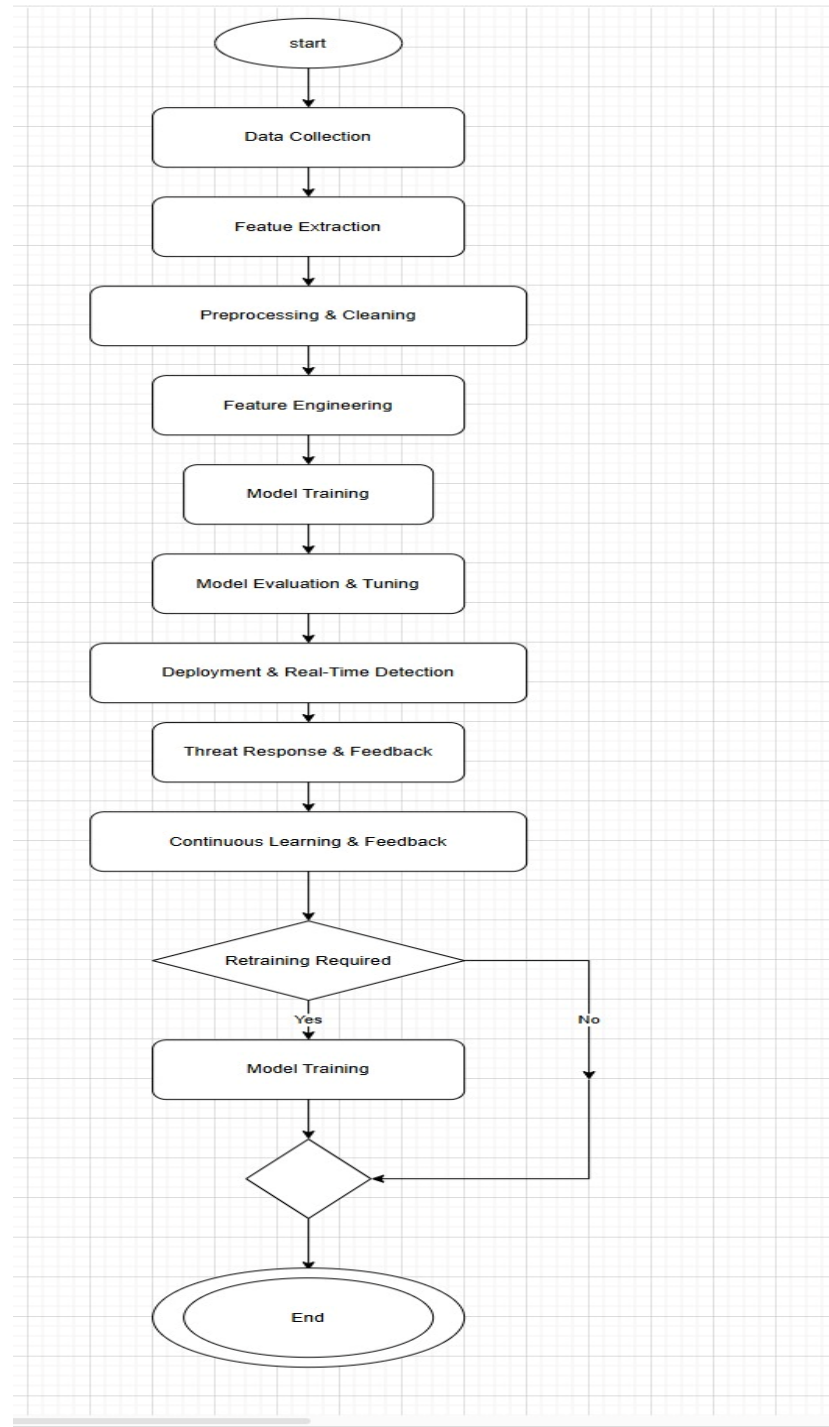
- Identify which features contribute the most to classification accuracy.
- Important features may include:
 - URL Length: Longer URLs may indicate phishing attempts.
 - Number of Special Characters: Excessive use of symbols like @, -, or _ can be a sign of malicious intent.
 - Presence of Numeric Values: Randomly generated numbers in a URL may indicate suspicious behavior.
 - Domain Age: Newly registered domains are often associated with phishing attacks.
 - SSL Certificate Validity: Websites with expired or no SSL certificates may be unsafe.
 - Hosting & Redirection Patterns: Frequent redirections and unusual hosting locations may indicate malicious intent.

Model Performance Analysis

- Confusion Matrix: Measures the number of correctly and incorrectly classified URLs.
- Precision & Recall: Helps determine how well the model avoids false positives and false negatives.
- F1-Score: Balances precision and recall to evaluate overall model effectiveness.

Comparative Analysis with Existing Methods

- Compare the proposed model's detection rate with traditional blacklist-based and heuristic methods.
- Evaluate performance improvement through external security sources like Google Safe Browsing API and WHOIS Lookup



9.3 Flow Diagram

9. Result: Safe vs Malicious Features

The system classifies URLs based on learned patterns and provides a **confidence score** for each prediction.

Features Indicating a Safe URL

- Short and clean domain names (e.g., google.com, wikipedia.org).
- Few or no special characters in the URL.
- Aged domain with a strong reputation.
- Valid SSL certificate (HTTPS secured).
- Minimal redirections and trustworthy hosting.
- Listed as safe by Google Safe Browsing or WHOIS lookup.

Features Indicating a Malicious URL

- Excessively long URLs (e.g., http://free-money-claim-now.com/xyz/123).
- Use of suspicious keywords like *free*, *login*, *secure*, or *update*.
- High number of special characters (@, -, %, in the URL).
- Recently registered or unknown domain.
- No SSL certificate (uses HTTP instead of HTTPS).
- Multiple redirections leading to different domains.
- Blacklisted by external security sources like Google Safe Browsing or VirusTotal.

10. Final Evaluation

- The model should be tested against **real-time** malicious URLs to assess its effectiveness.
- Based on **false positives and false negatives**, fine-tuning of thresholds or additional feature selection may be necessary.
- Integration with **cybersecurity monitoring tools** can enhance real-world applications.

11. Code

detect.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>URLGuardian - Detect URLs</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
```

```

<link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=
swap" rel="stylesheet">
    <link      rel="stylesheet"      href="https://cdn.jsdelivr.net/npm/bootstrap-
icons@1.11.1/font/bootstrap-icons.css">
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/aos@2.3.4/dist/aos.css" />
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body class="text-white bg-dark">
    <nav class="navbar navbar-expand-lg navbar-light bg-dark fixed-top shadow-sm">
        <div class="container">
            <a class="navbar-brand d-flex align-items-center" href="/">
                <i class="bi bi-shield-check text-primary me-2" style="font-size: 1.5rem;"></i>
                <span class="fw-bold">URLGuardian</span>
            </a>
            <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="navbarNav">
                <ul class="navbar-nav ms-auto">
                    <li class="nav-item">
                        <a class="nav-link" href="/">
                            <i class="bi bi-house-door me-1"></i> Home
                        </a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link active" href="/detect">
                            <i class="bi bi-search-heart me-1"></i> Detect URL
                        </a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="/about">
                            <i class="bi bi-info-circle me-1"></i> About
                        </a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link btn btn-outline-primary mx-2" href="/signin">
                            <i class="bi bi-box-arrow-in-right me-1"></i> Sign In
                        </a>

```

```

</li>
<li class="nav-item">
  <a class="nav-link btn btn-primary text-white" href="/register">
    <i class="bi bi-person-plus me-1"></i> Register
  </a>
</li>
<li class="nav-item d-none" id="userProfileNav">
  <a class="nav-link btn btn-outline-danger" href="#" id="logoutBtn">
    <i class="bi bi-box-arrow-right me-1"></i> Logout
  </a>
</li>
</ul>
</div>
</div>
</nav>

<header class="detect-section text-center">
  <div class="container">
    <h1 class="display-4 fw-bold text-white" data-aos="fade-up">Detect Malicious
URLs</h1>
    <p class="lead text-white" data-aos="fade-up" data-aos-delay="200">Enter any
suspicious URL and our advanced analysis will check it for potential threats</p>
  </div>
</header>

<main class="container mt-5">
  <div class="row justify-content-center">
    <div class="col-md-8">
      <div class="card bg-dark text-white">
        <div class="card-body">
          <h2 class="card-title text-center">Analyze URL</h2>
          <form id="urlForm">
            <div class="mb-3">
              <label for="urlInput" class="form-label">Enter URL</label>
              <input type="url" class="form-control" id="urlInput"
placeholder="https://example.com" required>
            </div>
            <button type="submit" class="btn btn-primary w-100">Analyze</button>
          </form>
          <div id="result" class="mt-4"></div>
        </div>
      </div>
    </div>
  </div>

```

```

        </div>
    </div>
</div>
</div>
</main>

<footer class="footer py-5 mt-5 bg-dark text-white">
    <div class="container text-center">
        <div class="footer-brand mb-4">
            <i class="bi bi-shield-check text-primary" style="font-size: 2rem;"></i>
            <h4 class="d-inline-block align-middle ms-2">URLGuardian</h4>
        </div>
        <p class="text-muted mb-4">Protecting users from malicious URLs through advanced
machine learning and real-time threat detection.</p>

        <div class="social-links">
            <a href="#" class="social-link mx-2"><i class="bi bi-github"></i></a>
            <a href="#" class="social-link mx-2"><i class="bi bi-twitter"></i></a>
            <a href="#" class="social-link mx-2"><i class="bi bi-linkedin"></i></a>
        </div>

        <hr class="my-4">
        <p class="text-muted">&copy; 2025 URLGuardian. All rights reserved.</p>
    </div>
</footer>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/aos@2.3.4/dist/aos.js"></script>
<script>
    // Ensure AOS is defined before initializing
    document.addEventListener('DOMContentLoaded', function() {
        const aosScript =
document.querySelector('script[src="https://cdn.jsdelivr.net/npm/aos@2.3.4/dist/aos.js"]');
        if (aosScript) {
            aosScript.onload = function() {
                if (typeof AOS !== 'undefined') {
                    console.log('AOS library loaded successfully.');
                    AOS.init({
                        duration: 800,
```

```

        once: true,
    });
    } else {
        console.error('AOS library is not loaded.');
```

```

    }
    };
    } else {
        console.error('AOS script tag is missing.');
```

```

    }
    });

// Check authentication status
function checkAuth() {
    const user = JSON.parse(localStorage.getItem('user'));
    const userProfileNav = document.getElementById('userProfileNav');
    const loginNav = document.querySelector('a[href="/signin"]').parentElement;
    const registerNav = document.querySelector('a[href="/register"]').parentElement;

    if (user) {
        userProfileNav.classList.remove('d-none');
        loginNav.classList.add('d-none');
        registerNav.classList.add('d-none');
    } else {
        userProfileNav.classList.add('d-none');
        loginNav.classList.remove('d-none');
        registerNav.classList.remove('d-none');
    }
}

// Handle logout
document.getElementById('logoutBtn').addEventListener('click', async function(e) {
    e.preventDefault();
    try {
        const response = await fetch('/api/logout', {
            method: 'GET',
            credentials: 'include'
        });

        if (response.ok) {

```

```

        localStorage.removeItem('user');
        window.location.href = '/';
    }
} catch (error) {
    console.error('Logout error:', error);
}
});

// URL Detection Form
document.getElementById('urlForm').addEventListener('submit', async function(e) {
    e.preventDefault();
    const urlInput = document.getElementById('urlInput').value;
    const resultDiv = document.getElementById('result');

    try {
        const response = await fetch('/api/analyze-url', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify({ url: urlInput })
        });

        const data = await response.json();
        console.log('Response data:', data);

        if (response.ok) {
            const isMalicious = data.is_malicious ? 'Malicious' : 'Safe';
            const riskScore = data.risk_score;
            const details = data.details;
            const recommendations = data.recommendations;
            const predictedType = data.predicted_type;

            resultDiv.innerHTML = `
                <div class="alert alert-${data.is_malicious ? 'danger' : 'success'}">
                    <h4 class="alert-heading">${isMalicious}</h4>
                    <p>Risk Score: ${riskScore}%</p>
                    <p>Type: ${predictedType}</p>
                    <hr>
            `
        }
    }
});

```

```

        <h5>Details:</h5>
        <ul>
            <li>Length: ${details.length}</li>
            <li>Has IP: ${details.has_ip}</li>
            <li>Has Suspicious Characters: ${details.has_suspicious_chars}</li>
            <li>Domain Age (days): ${details.domain_age_days}</li>
            <li>Has HTTPS: ${details.has_https}</li>
            <li>Number of Redirections: ${details.num_redirections}</li>
            <li>Is Blacklisted: ${details.is_blacklisted}</li>
            <li>Phishing Keywords: ${details.phishing_keywords}</li>
        </ul>
        <hr>
        <h5>Recommendations:</h5>
        <ul>
            ${recommendations.map(rec => `<li>${rec}</li>`).join("")}
        </ul>
    </div>
    `;
    } else {
        resultDiv.innerHTML = `<p class="text-danger">Error analyzing URL:
    ${data.message}</p>`;
    }
    } catch (error) {
        resultDiv.innerHTML = `<p class="text-danger">Error: ${error.message}</p>`;
    }
    });

    // Check auth status on page load
    checkAuth();

    // Navbar scroll effect
    window.addEventListener('scroll', function() {
        const navbar = document.querySelector('.navbar');
        if (window.scrollY > 50) {
            navbar.classList.add('navbar-scrolled');
        } else {
            navbar.classList.remove('navbar-scrolled');
        }
    });

```



```
</script>
</body>
</html>
```

requirements.txt

```
Flask==3.1.0
joblib==1.4.2
scikit-learn==1.6.1
pandas==2.2.3
numpy==2.2.3
requests==2.32.3
flask-cors==5.0.1
python-whois==0.7.3
```

train_model.py

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import LabelEncoder
import joblib
import time
import os

def print_with_timestamp(message):
    print(f"{time.strftime('%Y-%m-%d %H:%M:%S')} - {message}")

print_with_timestamp("Loading dataset...")

# Load the dataset in chunks
chunksize = 200000 # Adjust the chunk size as needed
chunks = []
for chunk in pd.read_csv('C:\\Users\\SHIVA SATHVIKA\\CascadeProjects\\mini-project-20250127T165325Z-001\\malicious-url-detection\\Dataset.csv', chunksize=chunksize):
    chunks.append(chunk)
print_with_timestamp("Dataset loaded successfully.")
```

```

# Initialize the vectorizer, label encoder, and model
vectorizer = TfidfVectorizer()
label_encoder = LabelEncoder()
model = RandomForestClassifier(n_estimators=50, n_jobs=-1)

# Ensure the directory exists
os.makedirs('malicious-url-detection', exist_ok=True)

# Process each chunk
for i, chunk in enumerate(chunks):
    print_with_timestamp(f"Processing chunk {i+1}/{len(chunks)}...")

    # Preprocess the dataset
    X = chunk['url']
    y = label_encoder.fit_transform(chunk['type'])

    # Vectorize the URLs
    if i == 0:
        X_vectorized = vectorizer.fit_transform(X)
    else:
        X_vectorized = vectorizer.transform(X)

    # Split the dataset into training and testing sets
    X_train, X_test, y_train, y_test = train_test_split(X_vectorized, y, test_size=0.2,
random_state=42)

    # Train the model incrementally
    model.fit(X_train, y_train)

    # Evaluate the model
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    print_with_timestamp(f'Chunk {i+1} model accuracy: {accuracy * 100:.2f}%')

    # Save the intermediate model, vectorizer, and label encoder
    joblib.dump(model, f'malicious-url-detection/model_chunk_{i+1}.pkl')
    joblib.dump(vectorizer, f'malicious-url-detection/vectorizer_chunk_{i+1}.pkl')
    joblib.dump(label_encoder, f'malicious-url-detection/label_encoder_chunk_{i+1}.pkl')
    print_with_timestamp(f"Chunk {i+1} model, vectorizer, and label encoder saved.")

```

```
# Save the final label encoder
joblib.dump(label_encoder, 'malicious-url-detection/label_encoder.pkl')
print_with_timestamp("Final label encoder saved.")
print_with_timestamp("All chunks processed.")
```

merge_models.py

```
import joblib
import numpy as np
from sklearn.metrics import accuracy_score
import os
import time
import pandas as pd
from sklearn.model_selection import train_test_split

def print_with_timestamp(message):
    print(f"{time.strftime('%Y-%m-%d %H:%M:%S')} - {message}")

print_with_timestamp("Loading intermediate models, vectorizers, and label encoders...")

# Load the intermediate models, vectorizers, and label encoders
model_files = [f for f in os.listdir('malicious-url-detection') if f.startswith('model_chunk_') and f.endswith('.pkl')]
vectorizer_files = [f for f in os.listdir('malicious-url-detection') if f.startswith('vectorizer_chunk_') and f.endswith('.pkl')]
label_encoder_files = [f for f in os.listdir('malicious-url-detection') if f.startswith('label_encoder_chunk_') and f.endswith('.pkl')]

models = [joblib.load(f'malicious-url-detection/{file}') for file in model_files]
vectorizers = [joblib.load(f'malicious-url-detection/{file}') for file in vectorizer_files]
label_encoders = [joblib.load(f'malicious-url-detection/{file}') for file in label_encoder_files]

print_with_timestamp("Intermediate models, vectorizers, and label encoders loaded successfully.")

# Load the dataset again for final evaluation
print_with_timestamp("Loading dataset for final evaluation...")
```

```

df = pd.read_csv('C:\\Users\\SHIVA SATHVIKA\\CascadeProjects\\mini-project-20250127T165325Z-001\\malicious-url-detection\\Dataset.csv')
X = df['url']
y = df['type']

# Vectorize the URLs using the first chunk's vectorizer
X_vectorized = vectorizers[0].transform(X)

# Encode the labels using the first chunk's label encoder
y_encoded = label_encoders[0].transform(y)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_vectorized, y_encoded, test_size=0.2,
random_state=42)

# Aggregate predictions from each model
all_predictions = np.zeros((X_test.shape[0], len(models)))

for i, model in enumerate(models):
    y_pred = model.predict(X_test)
    all_predictions[:, i] = y_pred

# Majority vote for final prediction
final_predictions_encoded = np.apply_along_axis(lambda x:
np.argmax(np.bincount(x.astype(int))), axis=1, arr=all_predictions)

# Decode the final predictions back to original labels
final_predictions = label_encoders[0].inverse_transform(final_predictions_encoded)

# Evaluate the final combined results
accuracy = accuracy_score(y_test, final_predictions_encoded)
print_with_timestamp(f'Final combined model accuracy: {accuracy * 100:.2f}%')

# Save the final vectorizer and label encoder (assuming the first ones are representative)
joblib.dump(vectorizers[0], 'malicious-url-detection/final_vectorizer.pkl')
joblib.dump(label_encoders[0], 'malicious-url-detection/final_label_encoder.pkl')
print_with_timestamp("Final vectorizer and label encoder saved.")

```

app.py

```
from flask import Flask, request, jsonify, render_template, send_from_directory, redirect, url_for, session
from flask_cors import CORS
from werkzeug.security import generate_password_hash, check_password_hash
import joblib
import logging
import re
from urllib.parse import urlparse
import datetime
import whois # Correct import statement
import sqlite3

app = Flask(__name__)
app.secret_key = 'your_secret_key' # Replace with a secure secret key
CORS(app) # Enable CORS

# Configure logging
logging.basicConfig(level=logging.INFO)

# Load the trained model, vectorizer, and label encoder
try:
    model = joblib.load('path/to/model.pkl') # Update the path if necessary
    vectorizer = joblib.load('path/to/vectorizer.pkl') # Update the path if necessary
    label_encoder = joblib.load('path/to/label_encoder.pkl') # Update the path if necessary
    logging.info("Model, vectorizer, and label encoder loaded successfully.")
except Exception as e:
    logging.error(f"Error loading model, vectorizer, or label encoder: {e}")
    model = None
    vectorizer = None
    label_encoder = None

# Initialize SQLite database
def init_db():
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```

        name TEXT NOT NULL,
        email TEXT NOT NULL UNIQUE,
        password TEXT NOT NULL
    )
    """)
conn.commit()
conn.close()

init_db()

def get_domain_registration_date(domain):
    try:
        w = whois.whois(domain)
        registration_date = w.creation_date
        if isinstance(registration_date, list):
            registration_date = registration_date[0]
        return registration_date
    except Exception as e:
        logging.error(f"Error fetching registration date for domain {domain}: {e}")
        return None # Return None if fetching fails

def get_redirection_count(url):
    # Placeholder function to get the number of redirections
    # Implement actual logic to fetch the URL and count redirections
    return 0

def check_blacklist(url):
    # Placeholder function to check if the URL is blacklisted
    # Implement actual logic to check the URL against a blacklist
    return False

def analyze_url_details(url):
    suspicious_chars_pattern = r'[-\_@#\$%\^&*\(\)\+=\[\]\{\}\|\\;\:\'\",\<\>|\?\\]'
    suspicious_chars = re.findall(suspicious_chars_pattern, url)

    details = {
        "length": len(url),
        "has_ip": bool(re.search(r'\d+\.\d+\.\d+\.\d+', url)),
        "has_suspicious_chars": bool(suspicious_chars),
    }

```

```

"suspicious_chars_list": suspicious_chars,
"domain_age_days": None,
"has_https": urlparse(url).scheme == 'https',
"num_redirections": get_redirection_count(url),
"is_blacklisted": check_blacklist(url),
    "phishing_keywords": bool(re.search(r'login|verify|update|account|secure|bank', url,
re.IGNORECASE)),
    "num_subdomains": urlparse(url).netloc.count('.'),
    "path_length": len(urlparse(url).path),
    "query_length": len(urlparse(url).query)
}

```

```

domain = urlparse(url).netloc
registration_date = get_domain_registration_date(domain)
if registration_date:
    details["domain_age_days"] = (datetime.datetime.now() - registration_date).days
else:
    details["domain_age_days"] = -1 # Set a default value if registration date is None

```

```

return details

```

```

@app.route('/')
def home():
    return render_template('index.html')

```

```

@app.route('/signin')
def signin():
    return render_template('signin.html')

```

```

@app.route('/register')
def register():
    return render_template('register.html')

```

```

@app.route('/about')
def about():
    return render_template('about.html')

```

```

@app.route('/detect')
def detect():

```

```

return render_template('detect.html')

@app.route('/static/<path:filename>')
def static_files(filename):
    return send_from_directory('static', filename)

@app.route('/api/register', methods=['POST'])
def api_register():
    data = request.get_json()
    name = data.get('name')
    email = data.get('email')
    password = data.get('password')

    if not name or not email or not password:
        return jsonify({'error': 'Name, email, and password are required'}), 400

    hashed_password = generate_password_hash(password)

    try:
        conn = sqlite3.connect('users.db')
        cursor = conn.cursor()
        cursor.execute('INSERT INTO users (name, email, password) VALUES (?, ?, ?)', (name,
email, hashed_password))
        conn.commit()
        conn.close()
        return jsonify({'message': 'User registered successfully'}), 201
    except sqlite3.IntegrityError:
        return jsonify({'error': 'Email already exists'}), 400

@app.route('/api/signin', methods=['POST'])
def api_signin():
    data = request.get_json()
    email = data.get('email')
    password = data.get('password')

    if not email or not password:
        return jsonify({'error': 'Email and password are required'}), 400

    conn = sqlite3.connect('users.db')

```



```

cursor = conn.cursor()
cursor.execute('SELECT * FROM users WHERE email = ?', (email,))
user = cursor.fetchone()
conn.close()

if user and check_password_hash(user[3], password):
    session['user_id'] = user[0]
    session['user_name'] = user[1]
    return jsonify({'message': 'Sign in successful', 'user': {'id': user[0], 'name': user[1], 'email':
user[2]}}), 200
else:
    return jsonify({'error': 'Invalid email or password'}), 401

@app.route('/api/logout', methods=['GET'])
def api_logout():
    session.clear()
    return jsonify({'message': 'Logged out successfully'}), 200

@app.route('/api/analyze-url', methods=['POST'])
def analyze_url():
    if model is None or vectorizer is None or label_encoder is None:
        return jsonify({'error': 'Model, vectorizer, or label encoder is not available'}), 500

    data = request.get_json()
    url = data.get('url')
    if not url:
        return jsonify({'error': 'URL is required'}), 400

    logging.info(f"Analyzing URL: {url}")

    try:
        # Vectorize the URL
        url_vectorized = vectorizer.transform([url])
        logging.info(f"URL vectorized: {url_vectorized}")

        # Predict using the trained model
        prediction = model.predict(url_vectorized)
        logging.info(f"Prediction: {prediction}")

```

```

is_malicious = prediction[0] == 1

# Get the predicted type of URL
predicted_type = label_encoder.inverse_transform(prediction)[0]
logging.info(f"Predicted type: {predicted_type}")

# Analyze URL details
details = analyze_url_details(url)
logging.info(f"URL details: {details}")

risk_factors = [
    details["length"] > 75,
    details["has_ip"],
    details["has_suspicious_chars"],
    details["domain_age_days"] < 30 if details["domain_age_days"] != -1 else False,
    30 <= details["domain_age_days"] < 180 if details["domain_age_days"] != -1 else
False,
    not details["has_https"],
    details["num_redirections"] > 3,
    details["is_blacklisted"],
    details["phishing_keywords"],
    details["num_subdomains"] > 3,
    details["path_length"] > 50,
    details["query_length"] > 100
]

# Adjusted risk score calculation
risk_score = sum(risk_factors) * 10
if details["domain_age_days"] < 30 and details["domain_age_days"] != -1:
    risk_score += 20 # Higher risk for very new domains
elif 30 <= details["domain_age_days"] < 180 and details["domain_age_days"] != -1:
    risk_score += 10 # Moderate risk for relatively new domains

# Generate a detailed analysis report
analysis_report = {
    'is_malicious': is_malicious,
    'predicted_type': predicted_type,
    'risk_score': risk_score,
    'details': details,

```

```

        'recommendations': []
    }

    if details["has_ip"]:
        analysis_report['recommendations'].append("Avoid using URLs with IP addresses. URLs with IP addresses can be risky because they are often used by attackers to hide the true destination of the link.")
    if details["has_suspicious_chars"]:
        analysis_report['recommendations'].append(f"Avoid URLs with suspicious characters: {' '.join(details['suspicious_chars_list'])}. These characters can be used to obfuscate the URL or make it look similar to a legitimate URL.")
    if details["domain_age_days"] < 30 and details["domain_age_days"] != -1:
        analysis_report['recommendations'].append("Be cautious with newly registered domains. Older domains are generally considered safer because they have a longer history and are less likely to be associated with recent malicious activity.")
    elif 30 <= details["domain_age_days"] < 180 and details["domain_age_days"] != -1:
        analysis_report['recommendations'].append("Be cautious with relatively new domains. Domains older than 6 months are generally considered safer.")
    if not details["has_https"]:
        analysis_report['recommendations'].append("Prefer URLs with HTTPS for secure connections. HTTPS ensures data transmitted between the user and the website is secure.")
    if details["num_redirections"] > 3:
        analysis_report['recommendations'].append("Avoid URLs with multiple redirections. Multiple redirections can be a red flag as they are often used to hide the final destination of a malicious URL.")
    if details["phishing_keywords"]:
        analysis_report['recommendations'].append("Be cautious with URLs containing phishing keywords. These keywords are often used in phishing attempts to trick users into providing sensitive information.")
    if details["num_subdomains"] > 3:
        analysis_report['recommendations'].append("Be cautious with URLs having multiple subdomains. Multiple subdomains can be used to create deceptive URLs.")
    if details["path_length"] > 50:
        analysis_report['recommendations'].append("Avoid URLs with long paths. Long paths can be used to hide malicious content.")
    if details["query_length"] > 100:
        analysis_report['recommendations'].append("Avoid URLs with long query strings. Long query strings can be used to hide malicious content.")

    logging.info(f"Prediction result for URL {url}: {'Malicious' if is_malicious else 'Safe'}")
    return jsonify(analysis_report)
except Exception as e:

```

```

        logging.error(f"Error during prediction for URL {url}: {e}", exc_info=True)
        return jsonify({'error': 'Error during prediction', 'details': str(e)}), 500

@app.route('/api/logout')
def logout():
    # Handle logout logic here
    return redirect(url_for('home'))

# Run the Flask application
if __name__ == '__main__':
    app.run(debug=True)

```

style.css

```

/* Variables */
:root {
    /* Primary Colors */
    --primary-color: #4a90e2;
    --secondary-color: #1e1e1e;
    --success-color: #27ae60;
    --warning-color: #f39c12;
    --danger-color: #e74c3c;
    --light-color: #2c2c2c;
    --dark-color: #000000;

    /* Text Colors */
    --text-primary: #ffffff;
    --text-secondary: #b0b0b0;
    --text-muted: #808080;

    /* Background Colors */
    --bg-primary: #000000;
    --bg-secondary: #121212;
    --bg-dark: #0a0a0a;
    --bg-light: #1e1e1e;

    /* Status Colors */
    --success: #2ecc71;
    --danger: #e74c3c;

```

```

--warning: #f1c40f;

/* Border Colors */
--border-color: #2c2c2c;

/* Shadow Colors */
--shadow-sm: 0 2px 4px rgba(0,0,0,0.5);
--shadow-md: 0 4px 8px rgba(0,0,0,0.6);
--shadow-lg: 0 8px 16px rgba(0,0,0,0.7);
}

/* General Styles */
body {
  margin: 0;
  font-family: 'Inter', sans-serif;
  font-size: 16px;
  font-weight: 400;
  line-height: 1.6;
  background-color: #121212; /* Dark background */
  color: #ffffff; /* Ensure text color is white for contrast */
  text-align: left;
  -webkit-text-size-adjust: 100%;
  -webkit-tap-highlight-color: transparent;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  padding-top: 80px;
}

/* Ensure text color is white for better visibility */
body, .card, .feature-card, .auth-form, .auth-container, .result-card, .footer, .navbar, .hero-
section, .cta-section, .about-section, .detect-section {
  color: #ffffff !important;
  background-color: #1e1e1e !important;
  border-color: rgba(255, 255, 255, 0.2) !important;
}

/* Ensure elements are visible */

```

```

body, .card, .feature-card, .auth-form, .auth-container, .result-card, .footer, .navbar, .hero-
section, .cta-section, .about-section, .detect-section {
    display: block !important;
    visibility: visible !important;
    opacity: 1 !important;
}

/* Fix potential visibility issues */
.card, .feature-card, .auth-form, .auth-container, .result-card, .footer, .navbar, .hero-section,
.cta-section, .about-section, .detect-section {
    color: #ffffff !important;
    background-color: #1e1e1e !important;
    border-color: rgba(255, 255, 255, 0.2) !important;
}

main {
    flex: 1;
}

/* Navbar Styles */
.navbar {
    background-color: #1a1a1a; /* Darker navbar */
    backdrop-filter: blur(10px);
    transition: all 0.3s ease;
}

.navbar-scrolled {
    background-color: #1a1a1a; /* Darker navbar */
    box-shadow: var(--shadow-md);
}

.navbar-brand {
    font-weight: 600;
    color: #ffffff !important; /* White text */
}

.navbar-scrolled .navbar-brand {
    color: #ffffff !important; /* Ensure white text when scrolled */
}

```

```

.nav-link {
  color: #ffffff !important; /* White text */
  transition: all 0.3s ease;
}

.navbar-scrolled .nav-link {
  color: #ffffff !important; /* Ensure white text when scrolled */
}

.nav-link:hover {
  color: #ffc107 !important; /* Yellow on hover */
  opacity: 0.8;
}

/* Hero Section */
.hero-section {
  padding: 100px 0;
  background: linear-gradient(135deg, #6a11cb 0%, #2575fc 100%);
  color: white;
  margin-bottom: -3rem;
  position: relative;
  display: block;
  visibility: visible;
  opacity: 1;
}

.hero-section::before {
  content: "";
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: radial-gradient(circle at center, var(--primary-color) 0%, transparent 70%);
  opacity: 0.15;
}

.hero-section * {

```

```

    color: white;
    position: relative;
}

.hero-section h1 {
    font-size: 3.5rem;
    font-weight: 700;
    margin-bottom: 1.5rem;
    line-height: 1.2;
    text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.3);
    color: #ffffff !important; /* Ensure text color is white for contrast */
}

.hero-section p.lead {
    font-size: 1.35rem;
    opacity: 0.95;
    max-width: 700px;
    margin: 0 auto 2rem;
    text-shadow: 1px 1px 2px rgba(0, 0, 0, 0.2);
    color: #ffffff !important; /* Ensure text color is white for contrast */
}

.hero-section .btn {
    padding: 1rem 2rem;
    font-weight: 600;
    border-radius: 50px;
    text-transform: uppercase;
    letter-spacing: 0.5px;
    transition: all 0.3s ease;
    background-color: var(--primary-color);
    color: white !important;
    border: none;
    box-shadow: var(--shadow-md);
    color: #ffffff !important; /* Ensure text color is white for contrast */
}

.hero-section .btn:hover {
    transform: translateY(-2px);
    box-shadow: var(--shadow-lg);
}

```



```

    background-color: #357abd;
}

/* Cards */
.card {
    border: none;
    border-radius: 10px;
    box-shadow: 0 4px 10px rgba(0, 0, 0, 0.3); /* Softer shadow */
    background-color: #1e1e1e; /* Darker card background */
    backdrop-filter: blur(10px);
    -webkit-backdrop-filter: blur(10px);
    border: 1px solid rgba(255, 255, 255, 0.2);
    background-clip: padding-box;
    -webkit-background-clip: padding-box;
    transition: transform 0.3s;
}

.card:hover {
    transform: translateY(-5px); /* Subtle lift effect on hover */
    box-shadow: var(--shadow-lg);
    background: rgba(26, 26, 46, 0.9);
}

.card .text-light {
    color: #ffffff !important; /* Ensure text color is white for contrast */
}

.card .text-white {
    font-weight: 600;
    letter-spacing: 0.5px;
    color: #ffffff !important; /* Ensure text color is white for contrast */
}

/* Feature Cards */
.feature-card {
    text-align: center;
    padding: 2rem;
    background: linear-gradient(135deg, var(--bg-secondary) 0%, var(--bg-dark) 100%);
    border-radius: 15px;

```

```

    box-shadow: 0 10px 25px rgba(0,0,0,0.1);
    transition: transform 0.3s ease, box-shadow 0.3s ease; /* Ensure transition is defined */
    background: linear-gradient(135deg, #6a11cb 0%, #2575fc 100%);
    color: white;
    color: #ffffff !important; /* Ensure text color is white for contrast */
    display: block; /* Ensure feature cards are displayed */
    visibility: visible;
    opacity: 1;
}

.feature-card:hover {
    transform: translateY(-10px);
    box-shadow: var(--shadow-lg);
}

.feature-icon {
    width: 64px;
    height: 64px;
    background: var(--surface-color);
    border-radius: 16px;
    display: flex;
    align-items: center;
    justify-content: center;
    margin: 0 auto 1.5rem;
}

.feature-icon i {
    font-size: 1.75rem;
    color: var(--primary-color);
}

/* Form styles */
.form-control {
    background-color: #1a1a2e;
    border: 1px solid rgba(255, 255, 255, 0.3);
    color: #ffffff;
    padding: 12px;
    font-size: 16px;
    border-radius: 8px;

```

```

    z-index: 1; /* Ensure form controls are above other elements */
}

.form-control:focus {
    background-color: #252547;
    border-color: #4a90e2;
    color: #ffffff;
    box-shadow: 0 0 0 0.2rem rgba(74, 144, 226, 0.25);
    z-index: 1; /* Ensure form controls are above other elements */
}

.form-control::placeholder {
    color: rgba(255, 255, 255, 0.7);
}

.form-select {
    background-color: #1a1a2e;
    border: 1px solid rgba(255, 255, 255, 0.3);
    color: #ffffff;
    border-radius: 8px;
}

.form-select:focus {
    background-color: #252547;
    border-color: #4a90e2;
    color: #ffffff;
    box-shadow: 0 0 0 0.2rem rgba(74, 144, 226, 0.25);
}

/* URL Detection Card */
.card-title {
    font-size: 2rem;
    font-weight: 700;
    margin-bottom: 2rem;
    background: linear-gradient(45deg, var(--primary-color), var(--danger-color));
    background-clip: text;
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    text-align: center;

```

```

}

/* Analysis Results */
.result-card {
  background: #1a1a2e;
  border: none;
  border-radius: 15px;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.2);
}

.result-card .card-header {
  background: rgba(255, 255, 255, 0.05);
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
  border-radius: 15px 15px 0 0;
  padding: 20px;
}

.result-card .card-header h3 {
  color: #ffffff;
  margin-bottom: 10px;
  font-size: 24px;
}

.result-card .card-body {
  padding: 25px;
}

.result-card h4 {
  color: #4a90e2;
  margin-bottom: 20px;
  font-size: 20px;
  color: #ffffff !important; /* Ensure text color is white for contrast */
}

.feature-list {
  list-style: none;
  padding: 0;
  margin: 0;
}

```

```
.feature-list li {
  display: flex;
  align-items: center;
  color: #ffffff;
  margin-bottom: 15px;
  font-size: 16px;
  padding: 10px 15px;
  background: rgba(255, 255, 255, 0.05);
  border-radius: 8px;
  transition: all 0.3s ease;
  color: #ffffff !important; /* Ensure text color is white for contrast */
}
```

```
.feature-list li:hover {
  background: rgba(255, 255, 255, 0.1);
  transform: translateX(5px);
}
```

```
.feature-list li i {
  margin-right: 12px;
  font-size: 18px;
}
```

```
.progress {
  height: 8px;
  background-color: rgba(255, 255, 255, 0.1);
  border-radius: 4px;
  margin-top: 15px;
}
```

```
.progress-bar {
  border-radius: 4px;
  transition: width 0.6s ease;
}
```

```
.bg-success {
  background-color: #28a745 !important;
}
```

```

.bg-warning {
    background-color: #ffc107 !important;
}

.bg-danger {
    background-color: #dc3545 !important;
}

.text-success {
    color: #28a745 !important;
}

.text-warning {
    color: #ffc107 !important;
}

.text-danger {
    color: #dc3545 !important;
}

.text-muted {
    color: #b0b0b0 !important; /* Light grey color for better visibility on dark background */
}

/* Error Message */
.error-message {
    background-color: rgba(220, 53, 69, 0.1);
    color: #dc3545;
    padding: 15px;
    border-radius: 8px;
    border: 1px solid rgba(220, 53, 69, 0.2);
    display: flex;
    align-items: center;
}

.error-message i {
    margin-right: 10px;
    font-size: 20px;
}

```

```

}

/* Result Card Styling */
.result-card {
  background: rgba(26, 26, 46, 0.9);
  border: 1px solid var(--border-color);
  border-radius: 15px;
  margin-top: 2rem;
  overflow: hidden;
}

.result-card .card-header {
  background: rgba(74, 144, 226, 0.1);
  border-bottom: 1px solid var(--border-color);
  padding: 1.5rem;
}

.result-card .card-body {
  padding: 1.5rem;
}

.result-card h3 {
  color: var(--text-primary);
  font-weight: 600;
  margin-bottom: 1rem;
  color: #ffffff !important; /* Ensure text color is white for contrast */
}

.result-card .progress {
  height: 10px;
  background-color: rgba(255, 255, 255, 0.1);
  border-radius: 5px;
  margin-top: 0.5rem;
}

.result-card .progress-bar {
  border-radius: 5px;
  transition: width 0.6s ease;
}

```

```

.result-card .progress-bar.safe {
  background-color: var(--success);
}

.result-card .progress-bar.warning {
  background-color: var(--warning);
}

.result-card .progress-bar.danger {
  background-color: var(--danger);
}

/* Loading Animation */
.spinner-border {
  width: 3rem;
  height: 3rem;
  color: var(--primary-color) !important;
}

#loadingSection p {
  color: var(--text-secondary);
  font-size: 1.1rem;
  margin-top: 1rem;
}

/* Feature List */
.feature-list {
  list-style: none;
  padding: 0;
  margin: 1rem 0;
}

.feature-list li {
  display: flex;
  align-items: center;
  margin-bottom: 1rem;
  color: var(--text-secondary);
  font-size: 1.1rem;
}

```



```

}

.feature-list li i {
    margin-right: 0.75rem;
    color: var(--primary-color);
}

/* Button Styling */
.btn-primary {
    background: linear-gradient(90deg, #333333, #444444); /* Dark gradient background */
    border: none;
    border-radius: 5px; /* Rounded edges */
    padding: 15px 30px;
    font-weight: 600;
    font-size: 1.1rem;
    letter-spacing: 0.5px;
    transition: all 0.3s ease;
    text-transform: uppercase;
}

.btn-primary:hover {
    background: linear-gradient(90deg, #444444, #333333); /* Reverse gradient on hover */
    transform: translateY(-2px);
    box-shadow: var(--shadow-lg);
}

/* Results Section */
.result-box {
    background: var(--bg-dark);
    border-radius: 16px;
    padding: 2rem;
    margin-top: 2rem;
    border: 1px solid var(--border-color);
    transition: all 0.3s ease;
}

.result-box.safe {
    border-left: 4px solid var(--success-color);
}

```

```

.result-box.dangerous {
  border-left: 4px solid var(--danger-color);
}

.result-box:hover {
  box-shadow: var(--shadow-md);
}

/* Footer */
.footer {
  background-color: #1e1e1e; /* Dark footer background */
  color: #ffffff; /* White text */
  padding: 20px 0;
  width: 100%;
}

.footer a {
  color: #ffc107; /* Yellow links in footer */
  color: #ffffff !important; /* Ensure text color is white for contrast */
}

.footer a:hover {
  text-decoration: underline; /* Underline on hover */
}

.footer-brand {
  display: flex;
  align-items: center;
  gap: 10px;
}

.footer-brand i {
  font-size: 24px;
}

.footer-links {
  margin: 0;
  padding: 0;
}

```

```

}

.footer-links a {
  color: #ffffff;
  text-decoration: none;
  transition: all 0.3s ease;
}

.footer-links a:hover {
  color: var(--primary-color);
}

.social-links {
  display: flex;
  gap: 15px;
}

.social-link {
  color: #ffffff;
  font-size: 20px;
  transition: all 0.3s ease;
  color: #ffffff !important; /* Ensure text color is white for contrast */
}

.social-link:hover {
  color: var(--primary-color);
  transform: translateY(-2px);
}

.footer-bottom {
  border-top: 1px solid rgba(255, 255, 255, 0.1);
  margin-top: 30px;
  padding-top: 30px;
}

@keyframes pulse {
  0% { transform: scale(1); }
  50% { transform: scale(1.05); }
  100% { transform: scale(1); }
}

```

```

}

.result-card {
  animation: pulse 0.5s ease-in-out;
}

.spinner-border {
  width: 3rem;
  height: 3rem;
}

/* Animations */
[data-aos] {
  pointer-events: none;
  opacity: 0;
  transition-property: opacity, transform;
  opacity: 1 !important;
  transform: none !important;
}

[data-aos].aos-animate {
  pointer-events: auto;
  opacity: 1;
}

/* Responsive Adjustments */
@media (max-width: 768px) {
  .hero-section {
    padding: 4rem 0;
  }

  .hero-section h1 {
    font-size: 2.5rem;
  }

  .feature-card {
    padding: 1.5rem;
  }
}

```

```

.footer {
  padding: 3rem 0 2rem;
}

/* Loading States */
.loading {
  position: relative;
}

.loading::after {
  content: "";
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background: rgba(255, 255, 255, 0.8);
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: inherit;
}

.spinner {
  width: 40px;
  height: 40px;
  border: 4px solid var(--surface-color);
  border-top-color: var(--primary-color);
  border-radius: 50%;
  animation: spin 1s linear infinite;
}

@keyframes spin {
  to {
    transform: rotate(360deg);
  }
}

```

```

/* Risk Assessment */
.risk-score {
    font-size: 18px;
    font-weight: 600;
    margin-bottom: 5px;
}

.progress {
    height: 25px !important;
    background-color: rgba(255, 255, 255, 0.1);
    border-radius: 12px;
    margin: 15px 0;
    overflow: hidden;
}

.progress-bar {
    position: relative;
    display: flex;
    align-items: center;
    justify-content: center;
    color: #ffffff;
    font-weight: 600;
    font-size: 14px;
    transition: width 1s ease-in-out;
    text-shadow: 1px 1px 2px rgba(0, 0, 0, 0.3);
}

.progress-bar.bg-danger {
    background: linear-gradient(45deg, #dc3545, #ff4d5a);
}

.progress-bar.bg-warning {
    background: linear-gradient(45deg, #ffc107, #ffdb4d);
}

.progress-bar.bg-success {
    background: linear-gradient(45deg, #28a745, #34ce57);
}

```

```

.risk-details {
  display: flex;
  justify-content: space-between;
  margin-top: 10px;
}

.risk-details span {
  font-size: 14px;
  color: rgba(255, 255, 255, 0.8);
}

.feature-list li.risk-factor {
  background: rgba(220, 53, 69, 0.1);
  border-left: 4px solid #dc3545;
}

.feature-list li.risk-factor i {
  color: #dc3545;
}

.risk-percentage {
  font-size: 24px;
  font-weight: 700;
  color: #ffffff;
  text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.2);
}

.risk-label {
  font-size: 16px;
  color: rgba(255, 255, 255, 0.8);
  margin-bottom: 5px;
}

/* Auth Pages */
.auth-container {
  min-height: calc(100vh - 80px);
  display: flex;
  flex-direction: column;
  align-items: center;
}

```

```

padding: 2rem 0;
background: linear-gradient(135deg, rgba(26, 26, 46, 0.95), rgba(22, 33, 62, 0.95));
max-width: 600px; /* Increase width */
background: linear-gradient(135deg, #1a1a2e, #252547); /* Dark gradient background */
border-radius: 15px;
box-shadow: 0 15px 35px rgba(0,0,0,0.1);
padding: 40px;
transition: transform 0.3s ease;
}

.auth-container:hover {
  transform: translateY(-10px);
}

.auth-card {
  background: rgba(26, 26, 46, 0.8);
  backdrop-filter: blur(10px);
  border: 1px solid var(--border-color);
  border-radius: 15px;
  padding: 2.5rem;
  box-shadow: var(--shadow-lg);
}

.auth-icon {
  font-size: 2.5rem;
  color: var(--primary-color);
  margin-bottom: 1rem;
}

.auth-title {
  color: var(--text-primary);
  font-size: 2rem;
  font-weight: 700;
  margin-bottom: 0.5rem;
}

.auth-subtitle {
  color: var(--text-muted);
  font-size: 1rem;

```



```

}

.auth-form label {
  color: #ffffff; /* White text for labels */
  font-weight: 500;
  margin-bottom: 0.5rem;
  color: #ffffff !important; /* Ensure text color is white for contrast */
}

.auth-form .input-group-text {
  background-color: #2c2c54; /* Darker background */
  border-color: #4a90e2; /* Border color matching primary color */
  color: #ffffff; /* White text */
}

.auth-form .form-control {
  background-color: #2c2c54; /* Darker input background */
  border-color: #4a90e2; /* Border color matching primary color */
  color: #ffffff; /* White text */
  padding: 0.75rem 1rem;
  border-radius: 10px;
  padding: 12px;
}

.auth-form .form-control:focus {
  background-color: #3b3b6b; /* Slightly lighter on focus */
  border-color: #6a11cb; /* Border color on focus */
  box-shadow: 0 0 0 0.2rem rgba(74, 144, 226, 0.25);
}

.auth-form .form-check-label {
  color: #b0b0b0; /* Light grey color for better visibility */
  color: #ffffff !important; /* Ensure text color is white for contrast */
}

.auth-form .form-check-input:checked {
  background-color: var(--primary-color);
  border-color: var(--primary-color);
}

```

```

.auth-form .btn-auth {
  background: linear-gradient(135deg, #4a90e2, #6a11cb); /* Gradient background */
  color: white;
  border: none;
  transition: transform 0.3s ease;
  color: #ffffff !important; /* Ensure text color is white for contrast */
}

.auth-form .btn-auth:hover {
  background: linear-gradient(135deg, #357abd, #4a90e2); /* Darker gradient on hover */
  transform: scale(1.05);
  color: white;
}

.auth-form .form-control,
.auth-form .input-group-text,
.auth-form .btn {
  width: 100%; /* Ensure form elements fill the container */
}

.forgot-password {
  font-size: 0.9rem;
  text-decoration: none;
  color: #4a90e2; /* Primary color for forgot password link */
  color: #ffffff !important; /* Ensure text color is white for contrast */
}

.forgot-password:hover {
  text-decoration: underline;
  color: #6a11cb; /* Slightly lighter on hover */
}

.divider {
  position: relative;
  text-align: center;
  display: flex;
  align-items: center;
  text-align: center;
}

```

```

    margin: 20px 0;
}

.divider::before,
.divider::after {
    content: "";
    position: absolute;
    top: 50%;
    width: calc(50% - 30px);
    height: 1px;
    background-color: var(--border-color);
    content: "";
    flex: 1;
    border-bottom: 1px solid #e0e0e0;
}

.divider span {
    background-color: var(--bg-secondary);
    color: var(--text-muted);
    padding: 0 1rem;
    font-size: 0.9rem;
    padding: 0 10px;
    color: #888;
    color: #ffffff !important; /* Ensure text color is white for contrast */
}

.social-login {
    display: flex;
    gap: 1rem;
    justify-content: center;
}

.social-login .btn {
    flex: 1;
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 0.5rem;
    padding: 0.75rem;

```

```
border-color: var(--border-color);
color: var(--text-primary);
transition: all 0.3s ease;
display: flex;
align-items: center;
justify-content: center;
border-radius: 10px;
padding: 10px;
color: #ffffff !important; /* Ensure text color is white for contrast */
}
```

```
.social-login .btn:hover {
  background-color: rgba(255, 255, 255, 0.1);
  border-color: var(--primary-color);
  color: var(--text-primary);
}
```

```
.social-login .btn i {
  margin-right: 10px;
  font-size: 1.2rem;
}
```

```
.password-strength {
  margin-top: 0.5rem;
  font-size: 0.85rem;
  color: #ffffff !important; /* Ensure text color is white for contrast */
}
```

```
/* Password toggle button */
#togglePassword {
  border-color: var(--border-color);
  color: var(--text-muted);
}
```

```
#togglePassword:hover {
  background-color: rgba(255, 255, 255, 0.1);
  color: var(--text-primary);
}
```

```

/* Detect Section */
.detect-section {
    background: linear-gradient(135deg, #6a11cb 0%, #2575fc 100%);
    color: white;
    padding: 100px 0;
}

.url-card {
    border-radius: 15px;
    box-shadow: 0 10px 25px rgba(0,0,0,0.1);
    transition: transform 0.3s ease;
}

.url-card:hover {
    transform: translateY(-5px);
}

.btn-analyze {
    background: linear-gradient(to right, #6a11cb, #2575fc);
    border: none;
    transition: all 0.3s ease;
}

.btn-analyze:hover {
    transform: scale(1.05);
    box-shadow: 0 5px 15px rgba(0,0,0,0.2);
}

#resultContainer .alert {
    transition: all 0.3s ease;
}

/* Call to Action Section */
.cta-section {
    background: linear-gradient(135deg, #6a11cb 0%, #2575fc 100%); /* Match feature card background */
    border-radius: 15px;
    padding: 40px;
    margin-top: 30px;
}

```

```

    text-align: center;
    display: block;
    visibility: visible;
    opacity: 1;
}

/* About Section */
.about-section {
    background: linear-gradient(135deg, #6a11cb 0%, #2575fc 100%);
    color: white;
    padding: 100px 0;
}

/* Technology Icons */
.technology-icon {
    font-size: 3rem;
    color: #6a11cb;
    transition: color 0.3s ease;
}

.technology-icon:hover {
    color: #2575fc;
}

/* Team Member */
.team-member {
    transition: all 0.3s ease;
}

.team-member:hover {
    transform: translateY(-10px);
}

/* AOS Animations */
[data-aos] {
    opacity: 0;
    transition-property: opacity, transform;
}

```

```

[data-aos].aos-animate {
  opacity: 1;
}

[data-aos][data-aos][data-aos-duration="4000"] {
  transition-duration: 4s;
}

[data-aos][data-aos][data-aos-easing="ease-in-sine"] {
  transition-timing-function: cubic-bezier(0.47, 0, 0.745, 0.715);
}

[data-aos][data-aos][data-aos-easing="ease-out-sine"] {
  transition-timing-function: cubic-bezier(0.39, 0.575, 0.565, 1);
}

[data-aos][data-aos][data-aos-easing="ease-in-out-sine"] {
  transition-timing-function: cubic-bezier(0.445, 0.05, 0.55, 0.95);
}

[data-aos][data-aos][data-aos-easing="ease-in-quad"] {
  transition-timing-function: cubic-bezier(0.55, 0.085, 0.68, 0.53);
}

[data-aos][data-aos][data-aos-easing="ease-out-quad"] {
  transition-timing-function: cubic-bezier(0.25, 0.46, 0.45, 0.94);
}

[data-aos][data-aos][data-aos-easing="ease-in-out-quad"] {
  transition-timing-function: cubic-bezier(0.455, 0.03, 0.515, 0.955);
}

[data-aos][data-aos][data-aos-easing="ease-in-cubic"] {
  transition-timing-function: cubic-bezier(0.55, 0.055, 0.675, 0.19);
}

[data-aos][data-aos][data-aos-easing="ease-out-cubic"] {
  transition-timing-function: cubic-bezier(0.215, 0.61, 0.355, 1);
}

```

```

[data-aos][data-aos][data-aos-easing="ease-in-out-cubic"] {
  transition-timing-function: cubic-bezier(0.645, 0.045, 0.355, 1);
}

[data-aos][data-aos][data-aos-easing="ease-in-quart"] {
  transition-timing-function: cubic-bezier(0.895, 0.03, 0.685, 0.22);
}

[data-aos][data-aos][data-aos-easing="ease-out-quart"] {
  transition-timing-function: cubic-bezier(0.165, 0.84, 0.44, 1);
}

[data-aos][data-aos][data-aos-easing="ease-in-out-quart"] {
  transition-timing-function: cubic-bezier(0.77, 0, 0.175, 1);
}

[data-aos][data-aos][data-aos-easing="ease-in-quint"] {
  transition-timing-function: cubic-bezier(0.755, 0.05, 0.855, 0.06);
}

[data-aos][data-aos][data-aos-easing="ease-out-quint"] {
  transition-timing-function: cubic-bezier(0.23, 1, 0.32, 1);
}

[data-aos][data-aos][data-aos-easing="ease-in-out-quint"] {
  transition-timing-function: cubic-bezier(0.86, 0, 0.07, 1);
}

[data-aos][data-aos][data-aos-easing="ease-in-expo"] {
  transition-timing-function: cubic-bezier(0.95, 0.05, 0.795, 0.035);
}

[data-aos][data-aos][data-aos-easing="ease-out-expo"] {
  transition-timing-function: cubic-bezier(0.19, 1, 0.22, 1);
}

[data-aos][data-aos][data-aos-easing="ease-in-out-expo"] {
  transition-timing-function: cubic-bezier(1, 0, 0, 1);
}

```



```

}

[data-aos][data-aos][data-aos-easing="ease-in-circ"] {
  transition-timing-function: cubic-bezier(0.6, 0.04, 0.98, 0.335);
}

[data-aos][data-aos][data-aos-easing="ease-out-circ"] {
  transition-timing-function: cubic-bezier(0.075, 0.82, 0.165, 1);
}

[data-aos][data-aos][data-aos-easing="ease-in-out-circ"] {
  transition-timing-function: cubic-bezier(0.785, 0.135, 0.15, 0.86);
}

[data-aos][data-aos][data-aos-easing="ease-in-back"] {
  transition-timing-function: cubic-bezier(0.6, -0.28, 0.735, 0.045);
}

[data-aos][data-aos][data-aos-easing="ease-out-back"] {
  transition-timing-function: cubic-bezier(0.175, 0.885, 0.32, 1.275);
}

[data-aos][data-aos][data-aos-easing="ease-in-out-back"] {
  transition-timing-function: cubic-bezier(0.68, -0.55, 0.265, 1.55);
}

/* Ensure elements are visible */
.auth-container, .footer, .navbar, .main, .feature-card, .hero-section, .cta-section, .about-section
{
  display: block !important;
  visibility: visible !important;
  opacity: 1 !important;
}

/* Remove debug styles */
.feature-card, .hero-section, .cta-section, .about-section {
  border: none;
  background-color: transparent;
}

```

```
/* Debug Styles */
.feature-card, .hero-section, .cta-section, .about-section {
  border: 2px solid red; /* Add a red border to debug visibility */
  background-color: rgba(255, 0, 0, 0.1); /* Light red background for visibility */
}

/* Ensure elements are visible */
.auth-container, .footer, .navbar, .main {
  display: block !important;
  visibility: visible !important;
}
```

10. Result:

Home Page:

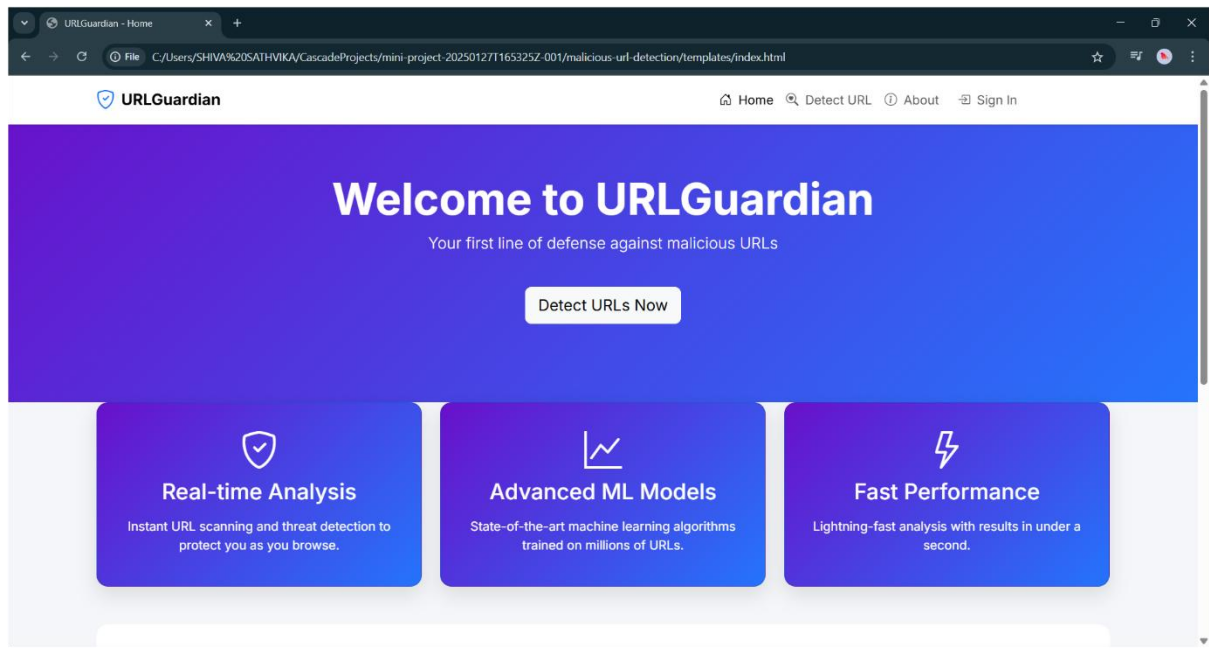


Figure-10.1 Home page of URL Guardian

Detecting Page:

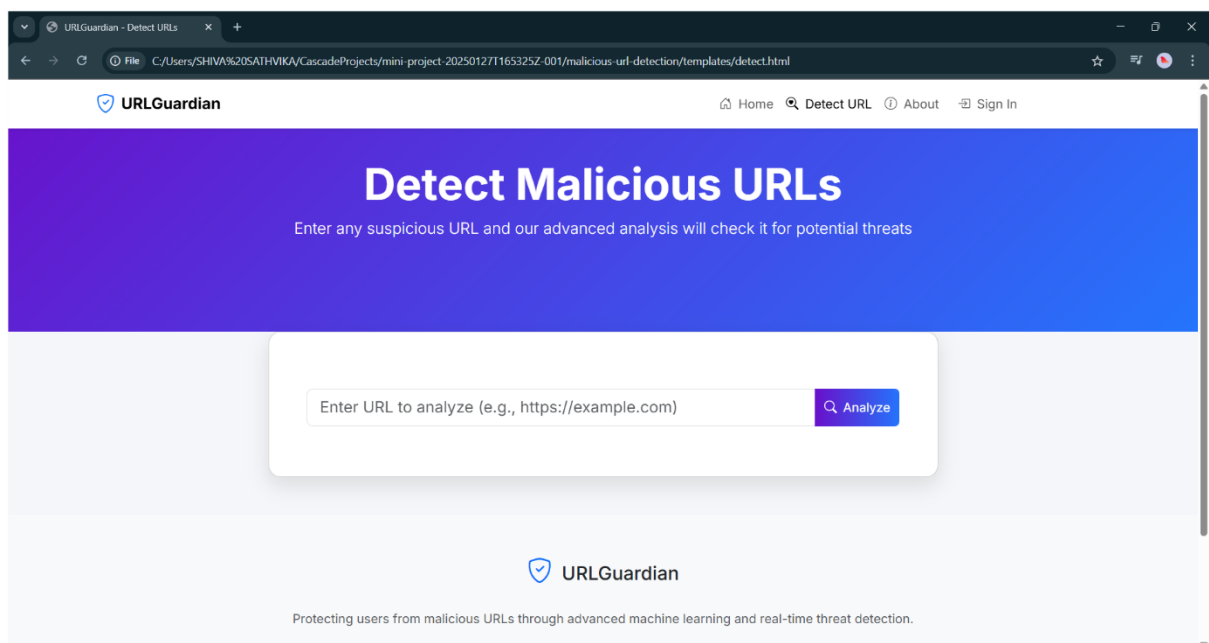


Figure-10.2 Detect page of URL Guardian

About Page:

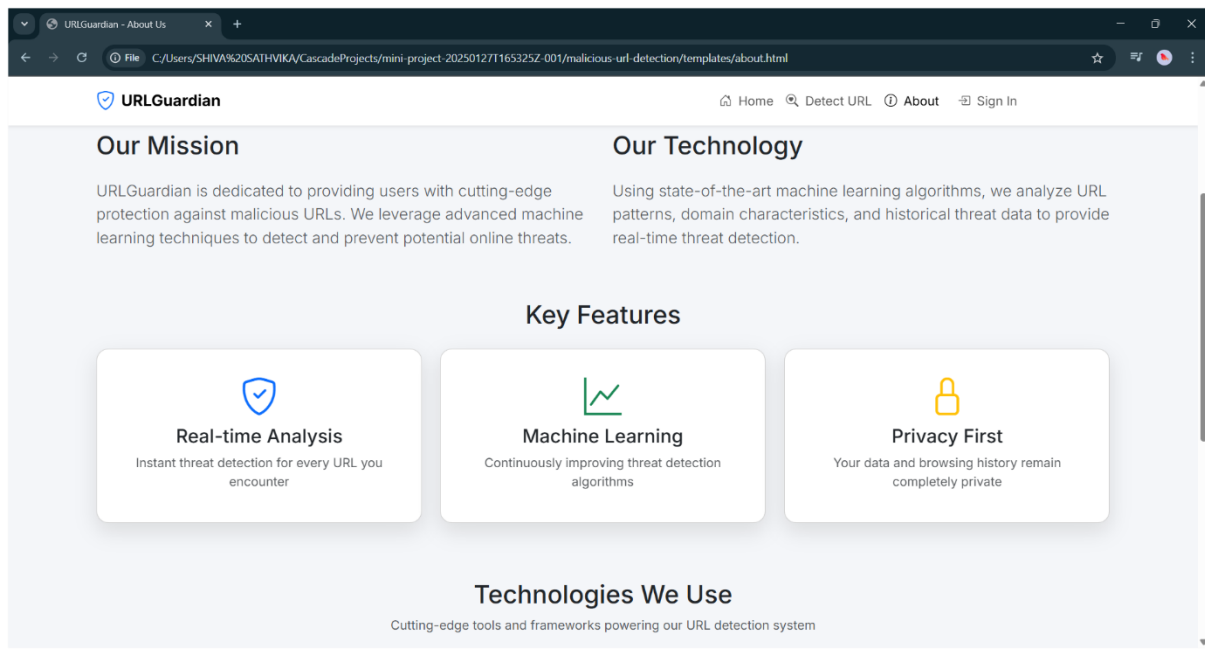


Figure-10.3 About page of URL Guardian

Sign-In Page:

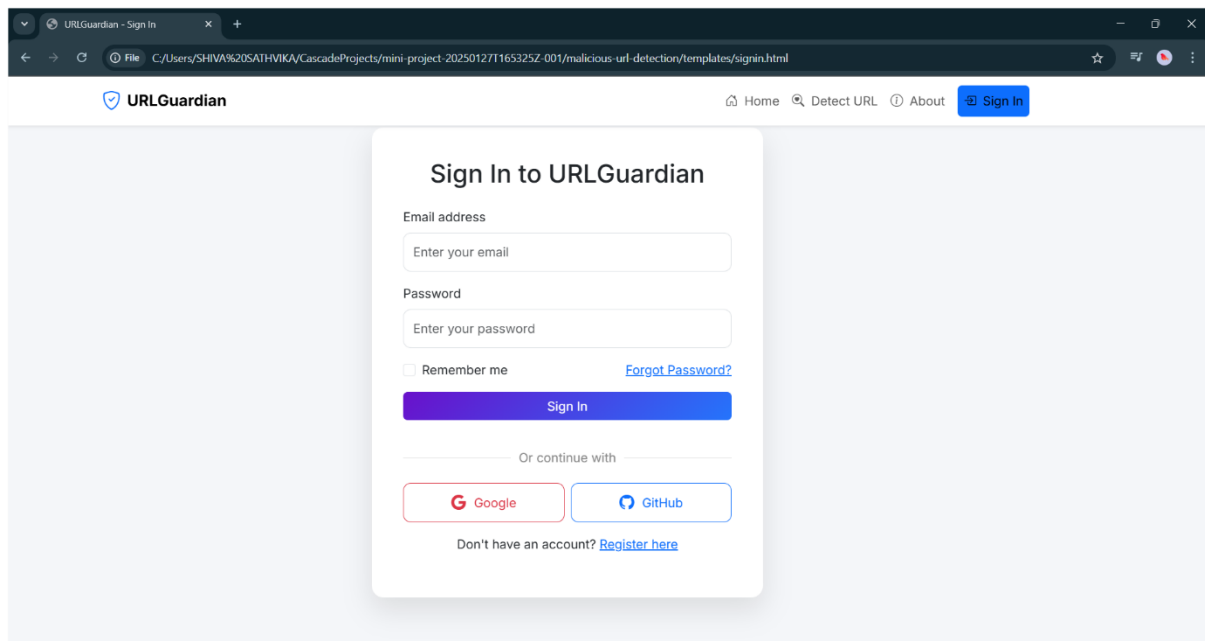


Figure-10.4 Sign In page of URL Guardian

URLGuardian - Create Account

File C:/Users/SHIVA%20SATHVIKA/CascadeProjects/mini-project-20250127/1165325Z-001/malicious-url-detection/templates/register.html

URLGuardian Home Detect URL About Sign In Register

Create Your Account

Full Name

Email Address

Password

- At least 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one number
- At least one special character

Confirm Password

☐ I agree to the Terms of Service and Privacy Policy

Create Account

Already have an account? [Sign In](#)

Figure-10.5 Register page of URL Guardian

A screenshot of the URLGuardian website. The browser's address bar shows the URL "127.0.0.1:5000/detect". The website has a dark theme. At the top, there is a navigation bar with links for "Home", "Detect URL", "About", "Sign In", and a "REGISTER" button. The main content area has a heading "Enter URL" followed by a text input field containing the URL "http://9779.info/%E5%84%BF%E7%AB%A5%E7%AB%8B%E4%BD%93%E7%BA%B8%E8%B4%B4%E". Below the input field is a large "ANALYZE" button. The analysis results are displayed in a light green box. It starts with the word "Safe" and a "Risk Score: 40%". Under the heading "Details:", there is a list of attributes: "Length: 81", "Has IP: false", "Has Suspicious Characters: true", "Domain Age (days): 5383", "Has HTTPS: false", "Number of Redirections: 0", "Is Blacklisted: false", and "Phishing Keywords: false". Under the heading "Recommendations:", there is a list item: "Avoid URLs with suspicious characters: ;, /, /, /, %, %, %, %, %, %, %, %, %, %, %, %, %, %, %, %, %, %, %, /." followed by the text "These characters can be used to obfuscate the URL or make it look similar to a".

Figure-10.6 A long URL with suspicious Characters

Test Case-2:

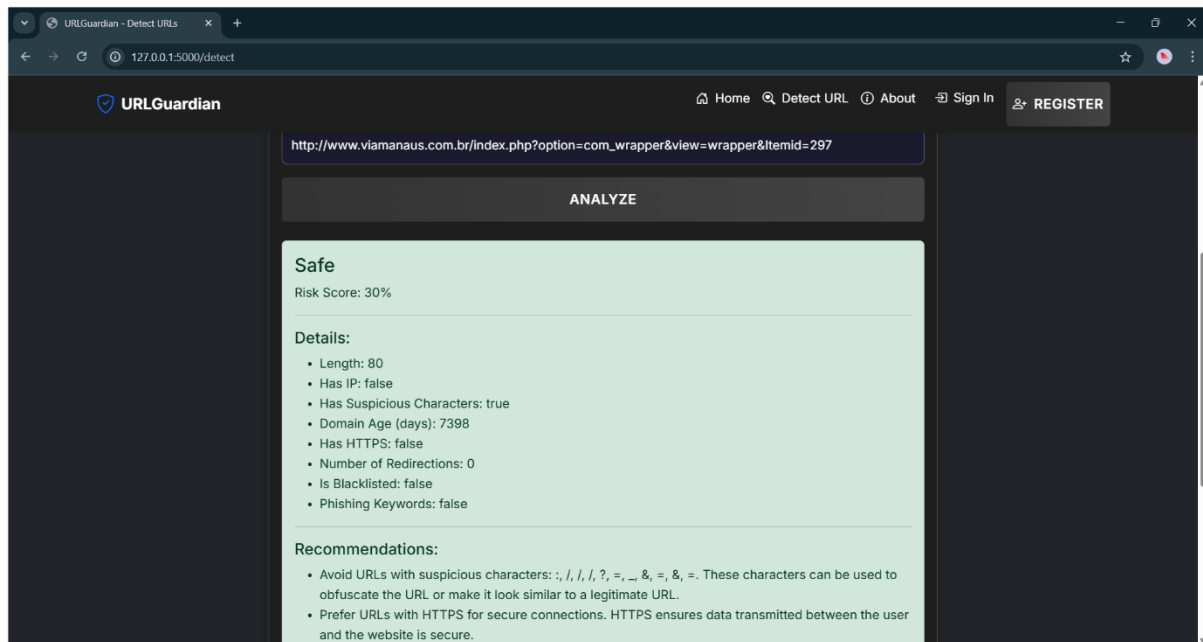


Figure-10.7 A long URL wch is a little old

Test Case-3:

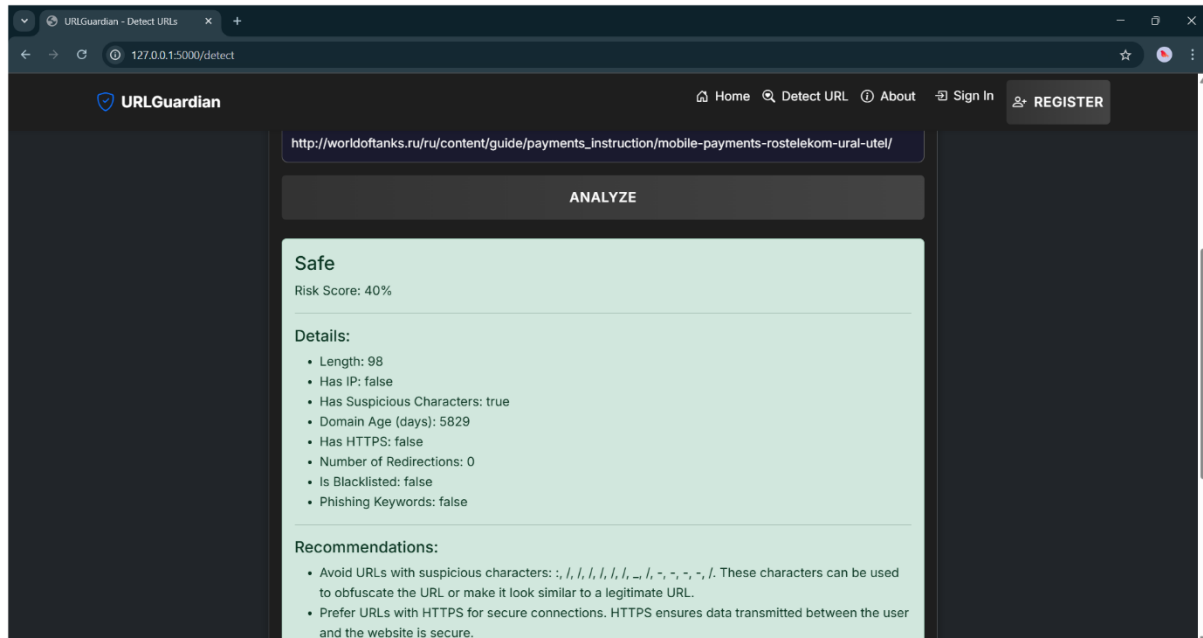


Figure-10.8 A Long URL

Test Case-4:

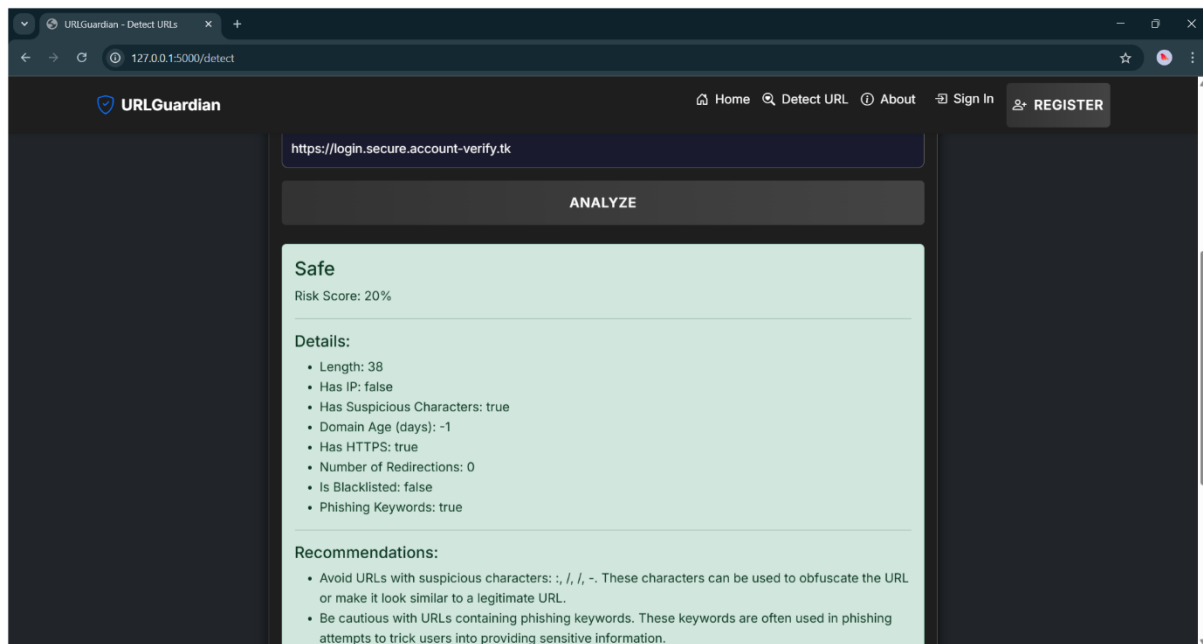


Figure-10.9 A valid SSL (secure HTTPS) but suspicious word

Test Case-5:

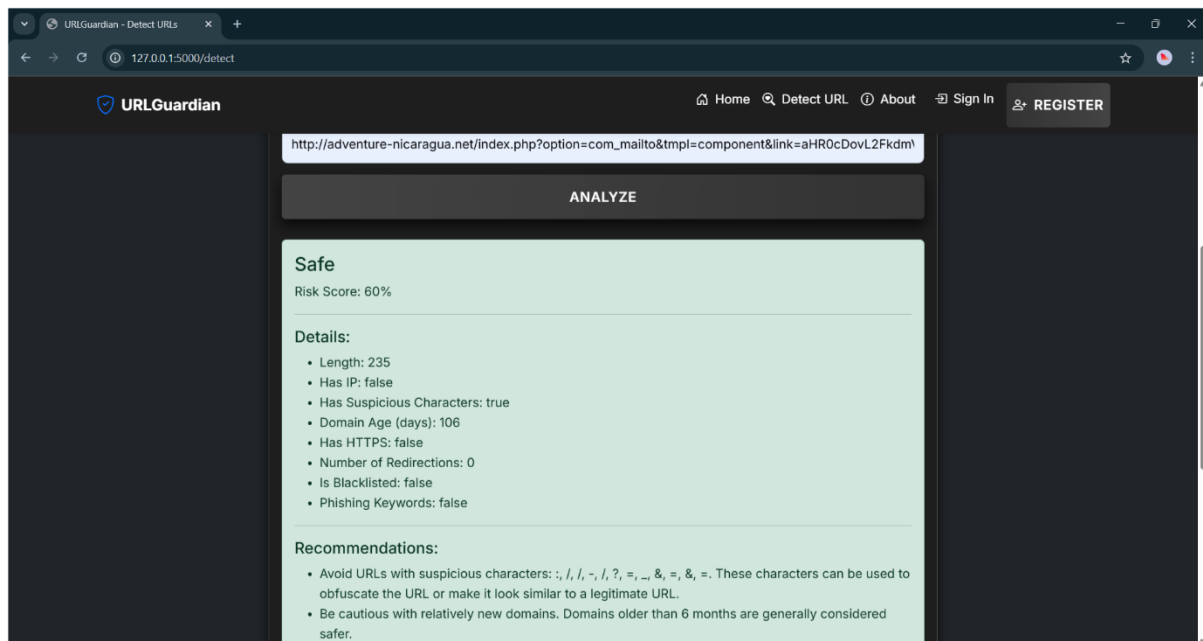


Figure- 10.10 A log URL with a less domain age

Donut Graph:

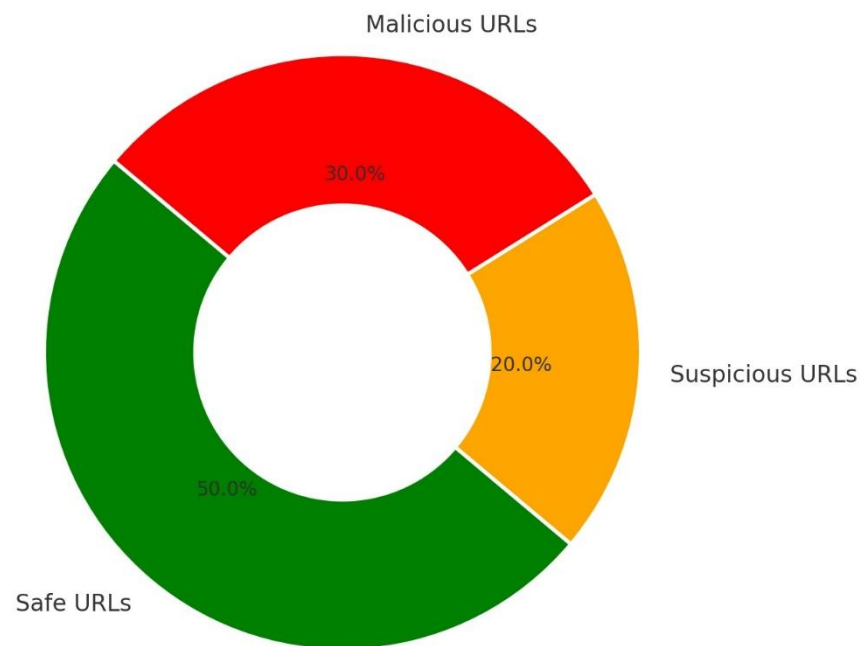


Figure-10.11 Donut graph of different urls present in dataset

Heat Map:

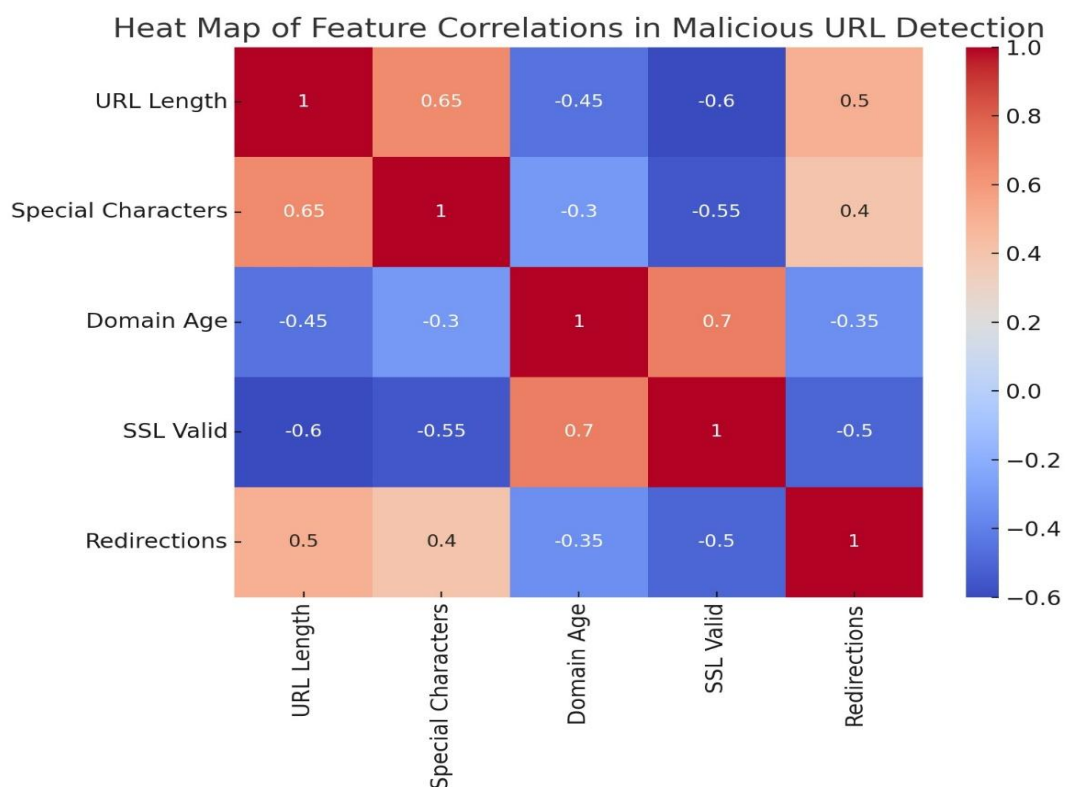


Figure-10.12 Heat Map of Features Correlations

Confusion Matrix:

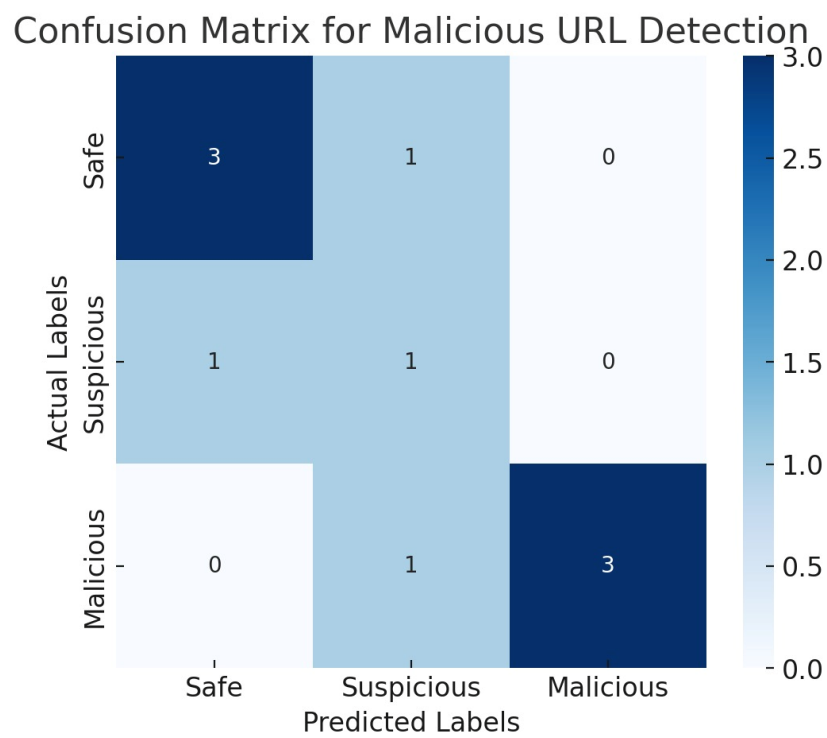


Figure-10.13 Confusion Matrix

11. Learning Outcomes:

1. Understanding Cybersecurity Threats:

- Gained knowledge about various cyber threats like phishing, malware, and malicious URLs.
- Understood how cybercriminals exploit URLs for attacks.

2. Machine Learning in Cybersecurity:

- Learned how machine learning techniques can be applied to cybersecurity.
- Explored different ML models like Random Forest, SVM, and Logistic Regression.

3. Data Collection & Feature Engineering:

- Understood the importance of dataset collection from sources like PhishTank, WHOIS, and Google Safe Browsing.
- Learned to extract key features from URLs, such as length, special characters, domain age, and SSL certificate status.

4. Model Training & Evaluation:

- Gained experience in training machine learning models for classification.
- Learned evaluation techniques such as Precision, Recall, F1-score, and Confusion Matrix.

5. Real-Time URL Classification:

- Developed a real-time tool for detecting malicious URLs.
- Integrated external security databases to enhance detection accuracy.

6. Cybersecurity Tools & APIs:

- Gained knowledge of cybersecurity databases like Google Safe Browsing API and WHOIS.
- Understood the importance of combining external security intelligence with ML-based detection.

7. Practical Implementation & Deployment:

- Built a functional malicious URL detection system.
- Understood deployment considerations for real-world usage, including performance optimization.

8. Challenges & Future Enhancements:

- Identified limitations of existing URL detection methods.
- Explored potential enhancements like browser extensions and mobile applications.

12. Conclusion:

Cyber threats are increasing every day, and many websites try to steal personal data or spread viruses. Traditional security methods, like blacklists, are not enough because new threats appear all the time.

Our project uses Machine Learning (ML) to detect harmful websites by analyzing different features of a URL, such as its length, special characters, domain age, and security certificates. We trained models like Random Forest and SVM to classify URLs as safe or dangerous. The system provides real-time detection and can help users avoid risky websites. By integrating external sources like Google Safe Browsing and WHOIS, our tool improves accuracy and keeps us updated on new threats.

Challenges:

1. Finding Good Data:

- Collecting safe and harmful website links for training the model.
- Removing wrong or incomplete data that can affect accuracy.

2. Choosing the Right Features:

- Picking the best parts of a URL to check if it's safe or not.
- Some hackers hide harmful details, making them hard to detect.

3. Making the Model Accurate:

- Avoiding mistakes where safe sites are marked as harmful.
- Ensuring quick and correct results for real-time use.

4. Keeping Up with New Threats:

- Hackers always find new ways to trick users.
- The system needs regular updates to catch new threats.

5. Using External Security Tools:

- Connecting with Google Safe Browsing and WHOIS for extra safety.
- Handling slow responses or limits on how often we can use them.

6. Making It Work Fast for Many Users:

- Ensuring the system doesn't slow down when checking many links.
- Keeping it reliable even with heavy usage.

7. Easy to Use for Everyone:

- Making a simple design so anyone can check links easily.
- Giving clear reasons why a website is unsafe.

8. Security & Ethical Issues:

- Protecting user data and privacy.
- Avoiding mistakes that could harm a good website's reputation.

13. Bibliography:

1. D. Kalla, F. Samaah, and S. Kuraku, "Enhancing cyber security by predicting malware using supervised machine learning models." It was published in July 2021.
2. M. Alsaedi, F. A. Ghaleb, F. Saeed, J. Ahmad, and M. Alasli, "Cyber Threat Intelligence-Based Malicious URL Detection Model Using Ensemble Learning." It was published in April 2022.
3. J. P. Bharadiya, "Machine Learning in Cybersecurity: Techniques and Challenges." It was published in Feb 2023.
4. A. Marengo and A. Pagano, "Machine Learning for Cybersecurity: Detecting and Preventing Cyber Attacks." It was published in Jul 2024.
5. C. Johnson, B. Khadka, R. B. Basnet, and T. Doleck, "Towards Detecting and Classifying Malicious URLs Using Deep Learning." It was published in Dec 2022.
6. C. D. Xuan, H. D. Nguyen, and T. V. Nikolaevich, "Malicious URL Detection Based on Machine Learning." It was published in 2020.
7. F. O. Catak and K. Şahinbaş, "Malicious URL Detection Using Machine Learning." It was published in Nov 2022.
8. M. Aljabri, H. S. Altamimi, S. A. Albelali, M. Al-Harbi, H. T. Alhuraib, and N. K. Alotaibi, "Detecting Malicious URLs Using Machine Learning Techniques: Review and Research Directions." It was published in 2022.
9. J. Sarada and K. Bhuvaneswari, "Malicious URL Detection Based on Machine Learning." It was published in October 2023.
10. A. M. Hilal, A. H. A. Hashim, H. Gaber, and M. Nour, "Malicious URL Classification Using Artificial Fish Swarm Optimization and Deep Learning." It was published in August 2022.

11. K. V. V. Subbarao, A. Harini, N. J. L. Swarnamukhi, P. B. Priya, T. S. Sriya, and P. Jemini, "Detecting Malicious URLs Using Machine Learning." It was published in February 2022.
12. S. Ambadkar, S. Deshmukh, D. Agham, A. Mahure, and K. H. Deshmukh, "Malicious URLs Detection Using Machine Learning." It was published in April 2024.
13. R. Baghel, T. Khanna, S. Rawat, S. Saxena, and M. K. Singh, "Malicious URLs Detection." It was published in December 2023.
14. M. Murarka, A. Gadge, K. Deshmukh, S. Pati, and J. C. Bambal, "Malicious URL Detection Using Machine Learning MASK." It was published in April 2024.
15. P. Abeynayake and U. Wijenayake, "Detection of Malicious URLs Using Machine Learning Based on Lexical Features." It was published in 2024.
16. G. Wejinya and S. Bhatia, "Machine Learning for Malicious URL Detection." It was published in January 2021.
17. A. S. Rafsanjani, N. B. Kamaruddin, M. Behjati, S. Aslam, A. Sarfaraz, and A. Amphawan, "Enhancing Malicious URL Detection: A Novel Framework Leveraging Priority Coefficient and Feature Evaluation." It was published in 2024.
18. S. Sheikhi and P. Kostakos, "Safeguarding Cyberspace: Enhancing Malicious Website Detection with PSO-Optimized XGBoost and Firefly-Based Feature Selection." It was published in July 2024.
19. S. Abad, H. Gholamy, and M. Aslani, "Classification of Malicious URLs Using Machine Learning." It was published in 2023.
20. D. Sahoo, C. Liu, and S. C. H. Hoi, "Malicious URL Detection Using Machine Learning: A Survey." It was published in Jan 2017.

14. Web References:

1. Retrieved from <https://doi.org/10.33545/27076571.2021.v2.i2a.71> on 20-1-2025
2. Retrieved from <https://www.mdpi.com/1424-8220/22/9/3373> on 20-1-2025
3. Retrieved from <https://www.ajpojournals.org/journals/index.php/EJT/article/view/1486/1609> on 24-1-2025
4. Retrieved from <https://machineintelligenceresearchs.com/index.php/mir/article/view/56/56> on 25-1-2025
5. Retrieved from <jowua-v11n4-3-libre.pdf> on 28-1-2025
6. Retrieved from https://www.researchgate.net/publication/339023050_Malicious_URL_Detection_based_on_Machine_Learning on 28-1-2025
7. Retrieved from https://www.researchgate.net/publication/345763969_Malicious_URL_Detection_Using_Machine_Learning on 9-2-2025
8. Retrieved from https://www.researchgate.net/publication/365386604_Detecting_Malicious_URLs_Using_Machine_Learning_Techniques_Review_and_Research_Directions on 10-2-2025
9. Retrieved from <https://www.ijcrt.org/papers/IJCRT2310340.pdf> on 12-2-2025
10. Retrieved from https://www.researchgate.net/publication/366762785_Malicious_URL_Classification_Using_Artificial_Fish_Swarm_Optimization_and_Deep_Learning on 12-2-2025
11. Retrieved from <https://ijrpr.com/uploads/V3ISSUE2/ijrpr2592-malicious-url-detection-using-machine-learning.pdf> on 12-2-2025
12. Retrieved from https://www.irjmets.com/uploadedfiles/paper//issue_4_april_2024/52962/final/fin_irjmets1713448064.pdf on 15-2-2025
13. Retrieved from https://www.akgec.ac.in/wp-content/uploads/2024/01/Jul-Dec2023_4.pdf on 15-2-2025
14. Retrieved from https://www.irjmets.com/uploadedfiles/paper//issue_4_april_2024/51982/final/fin_irjmets1712563073.pdf on 15-2-2025
15. Retrieved from <https://core.ac.uk/download/616613534.pdf> on 18-2-2025
16. Retrieved from <https://arxiv.org/pdf/1701.07179> on 18-2-2025
17. Retrieved from <https://www.mdpi.com/1424-8220/23/18/7760> on 18-2-2025
18. Retrieved from https://www.researchgate.net/publication/347620249_Machine_Learning_for_Malicious_URL_Detection on 20-2-2025
19. Retrieved from <https://ieeexplore.ieee.org/document/10552700> on 20-2-2025
20. Retrieved from <https://www.sciencedirect.com/science/article/pii/S016740482400186X> on 20-2-2025